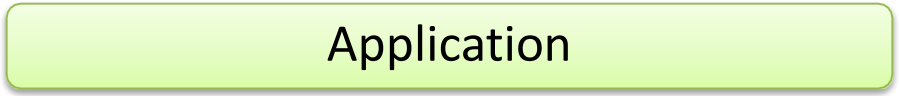
Advanced Computer Architecture CMP 511

Introduction to Parallel Processing

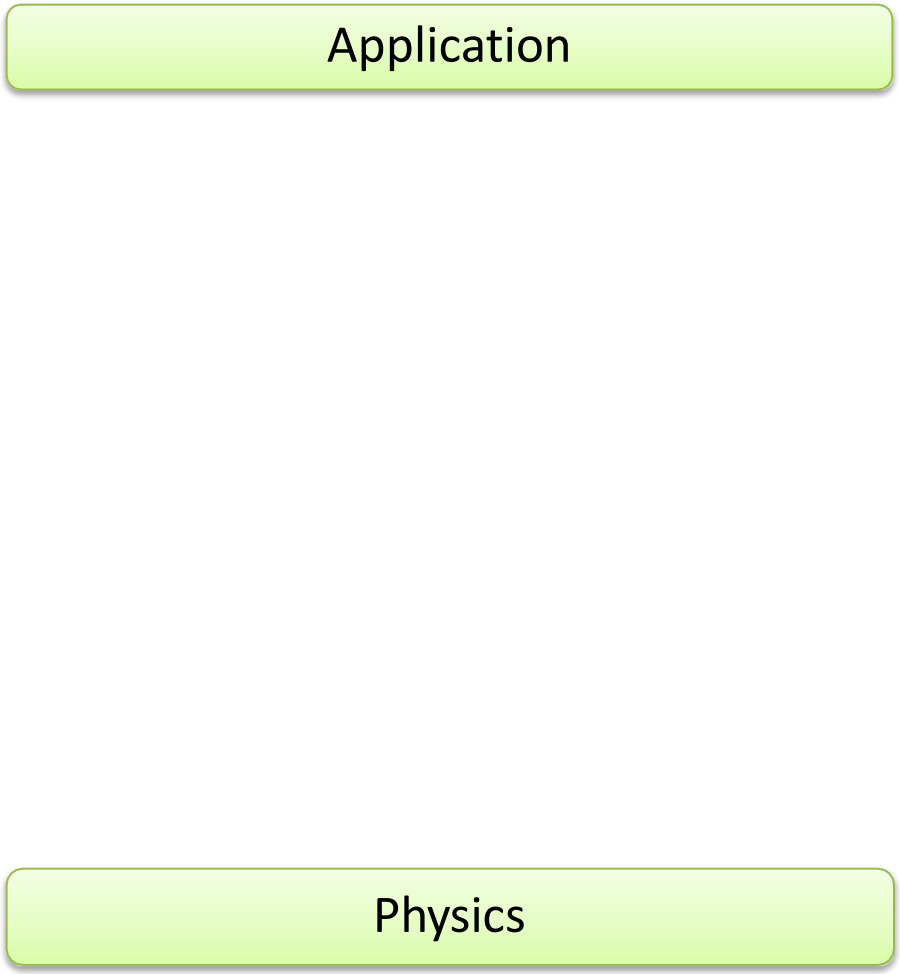
Department of Computer Engineering

Pokhara University

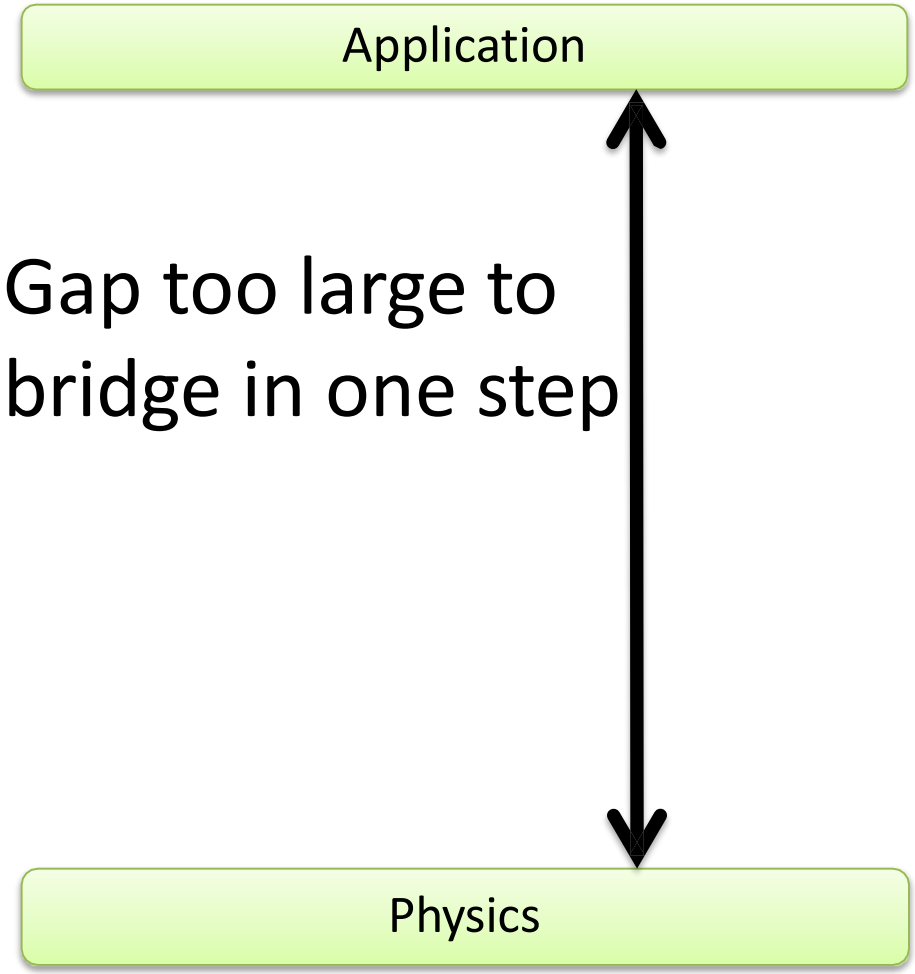
What is Computer Architecture?



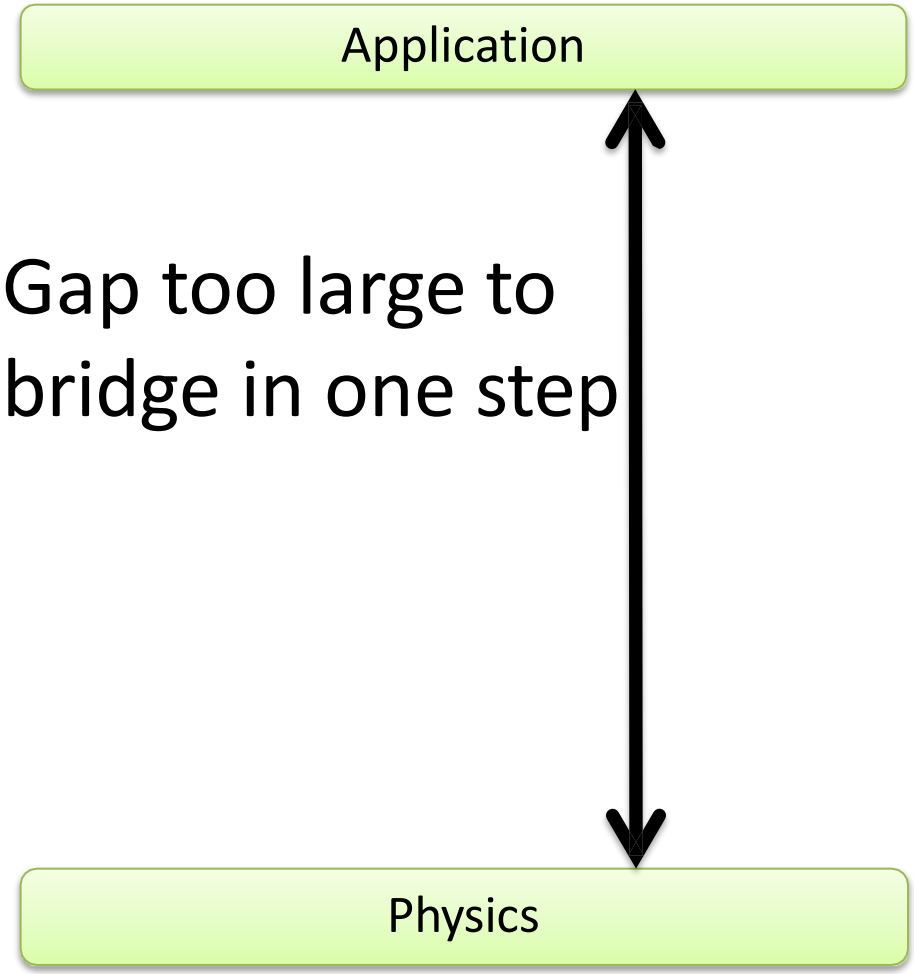
What is Computer Architecture?



What is Computer Architecture?

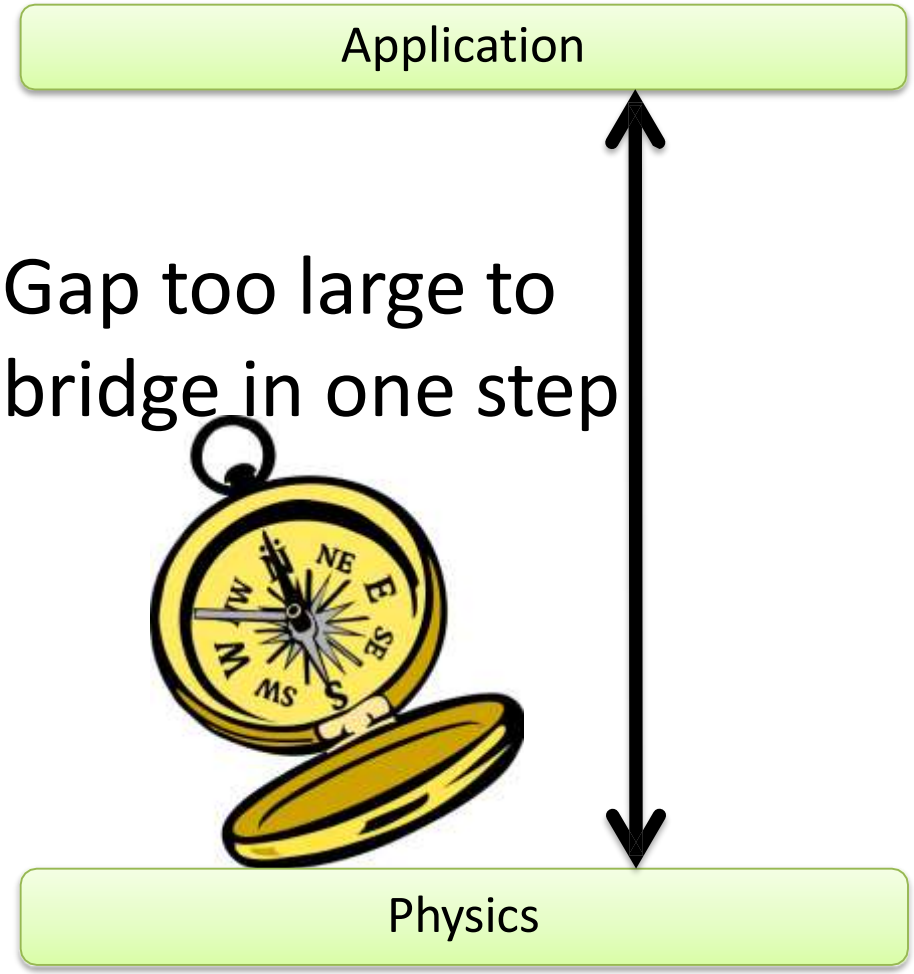


What is Computer Architecture?



In its broadest definition, computer architecture is the design of the abstraction/implementation layers that allow us to execute information processing applications efficiently using manufacturing technologies

What is Computer Architecture?

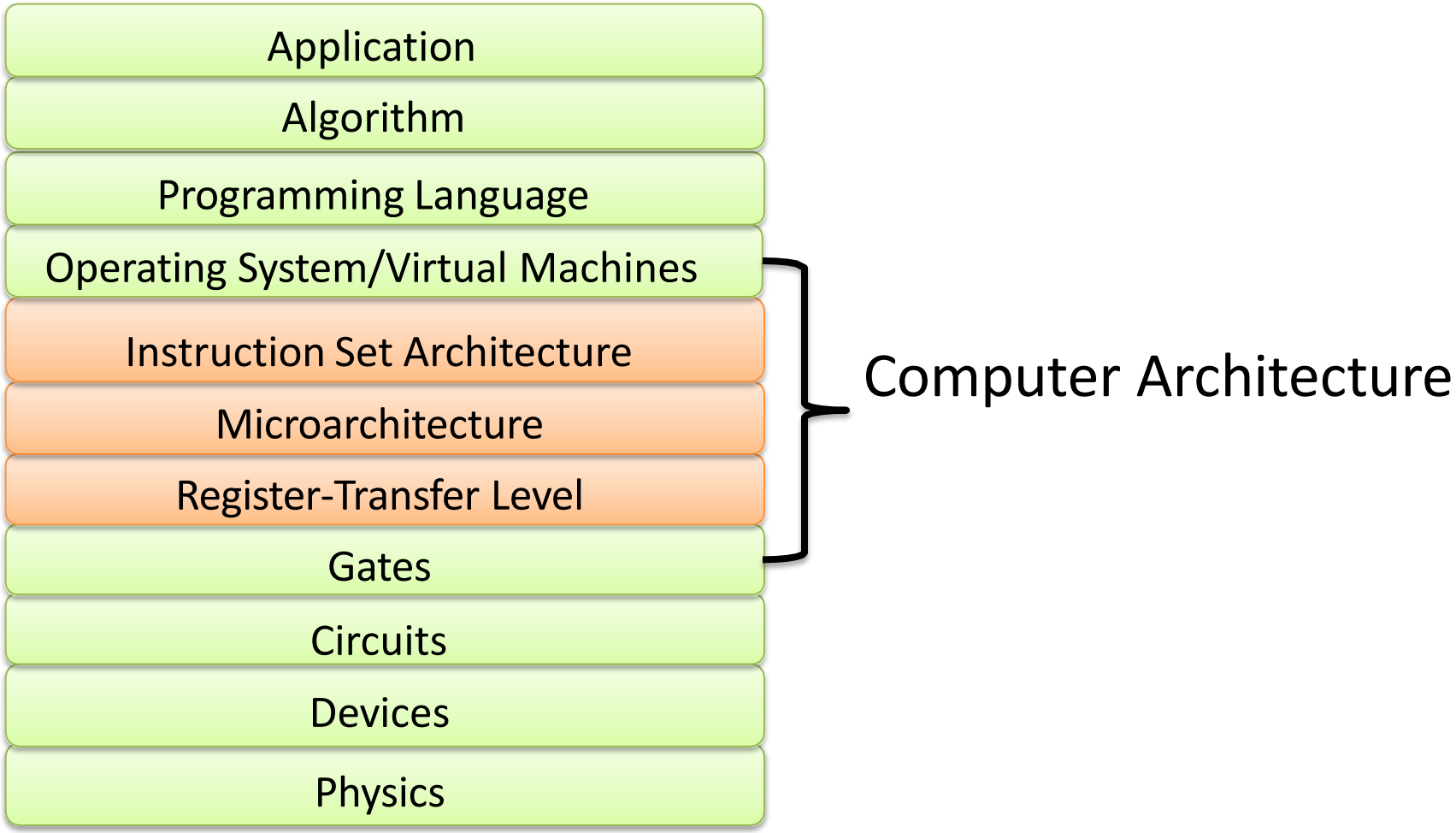


In its broadest definition, computer architecture is the design of the abstraction/implementation layers that allow us to execute information processing applications efficiently using manufacturing technologies

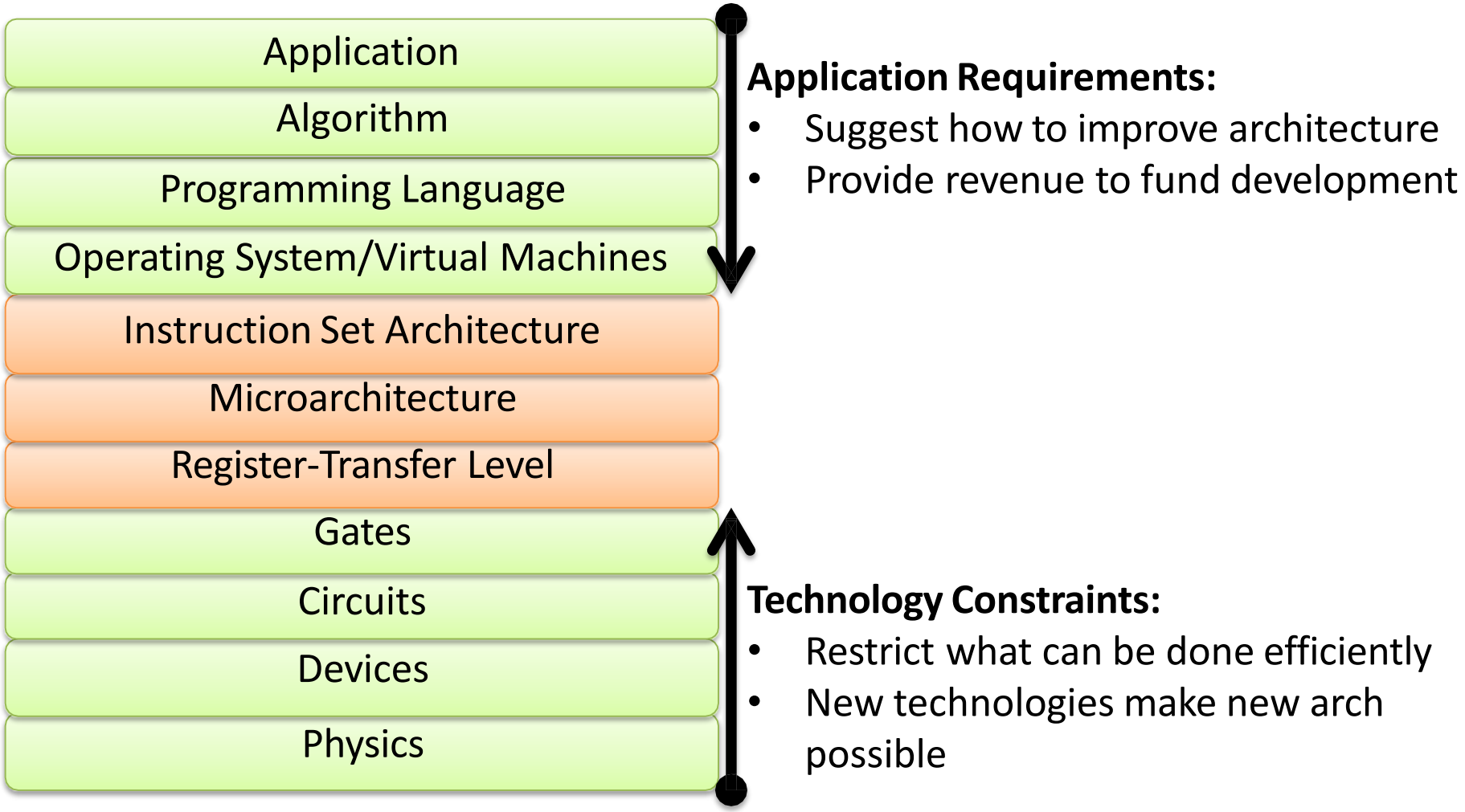
Abstractions in Modern Computing Systems



Abstractions in Modern Computing Systems



Computer Architecture is Constantly Changing



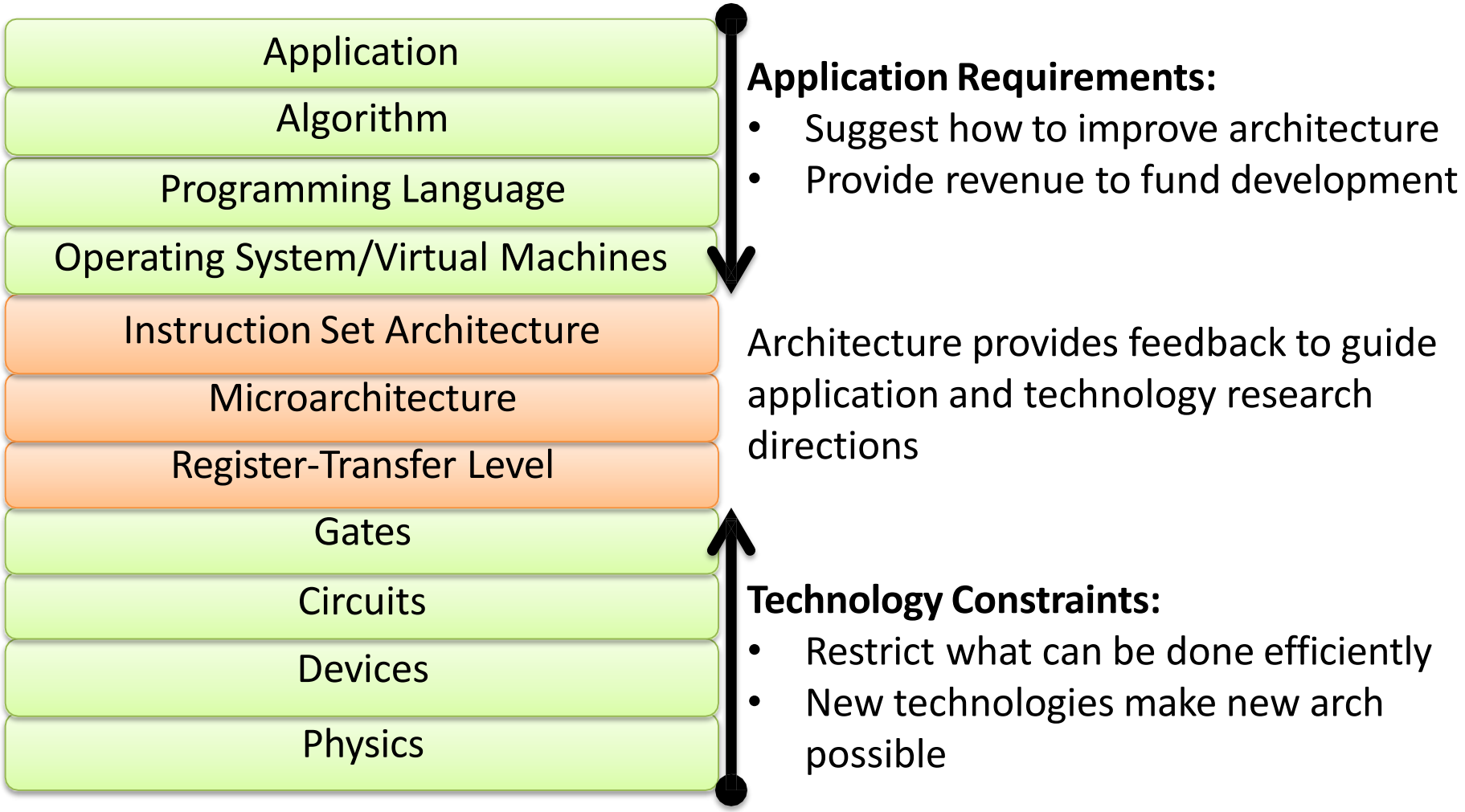
Application Requirements:

- Suggest how to improve architecture
- Provide revenue to fund development

Technology Constraints:

- Restrict what can be done efficiently
- New technologies make new arch possible

Computer Architecture is Constantly Changing



Application Requirements:

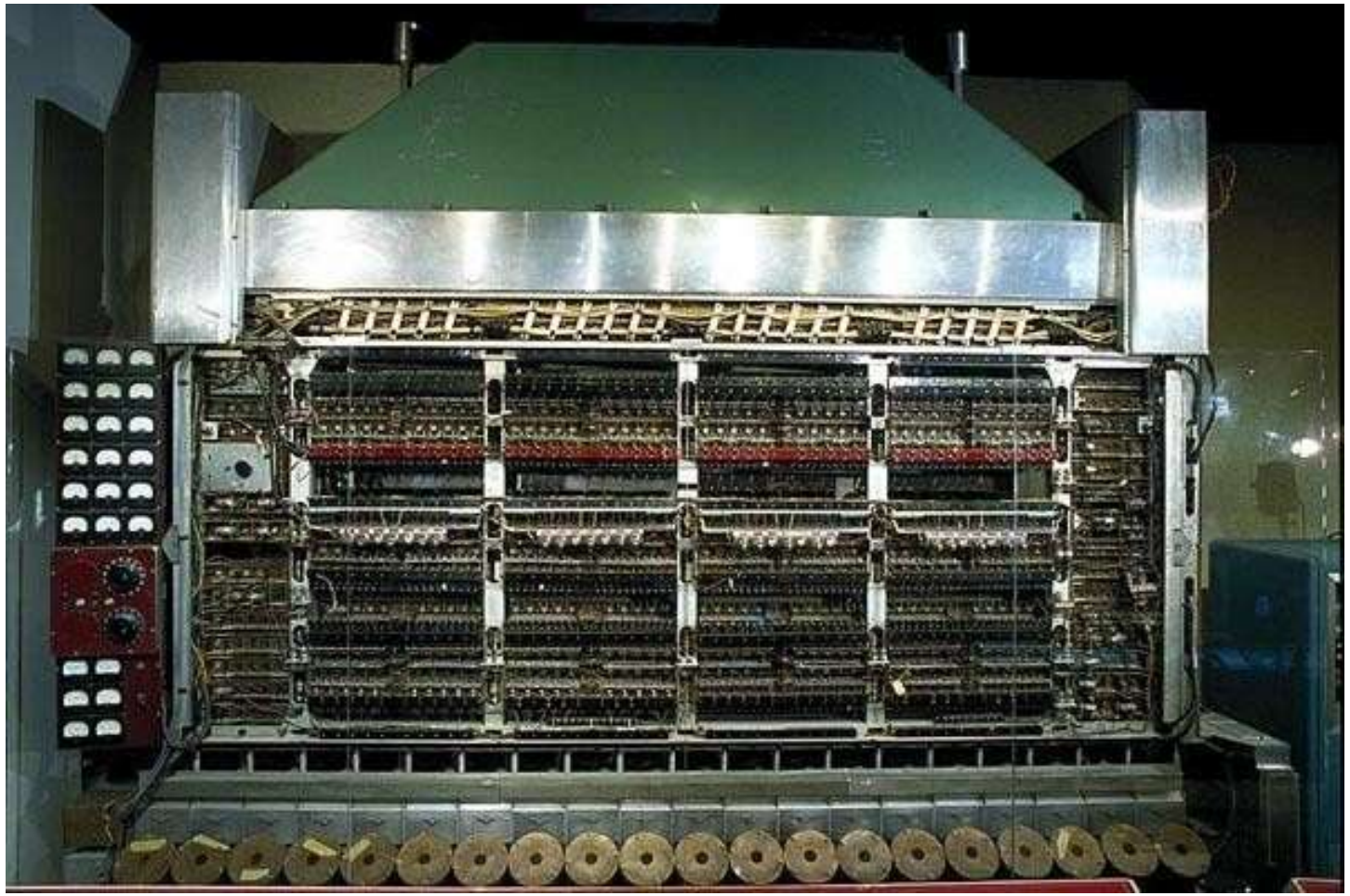
- Suggest how to improve architecture
- Provide revenue to fund development

Architecture provides feedback to guide application and technology research directions

Technology Constraints:

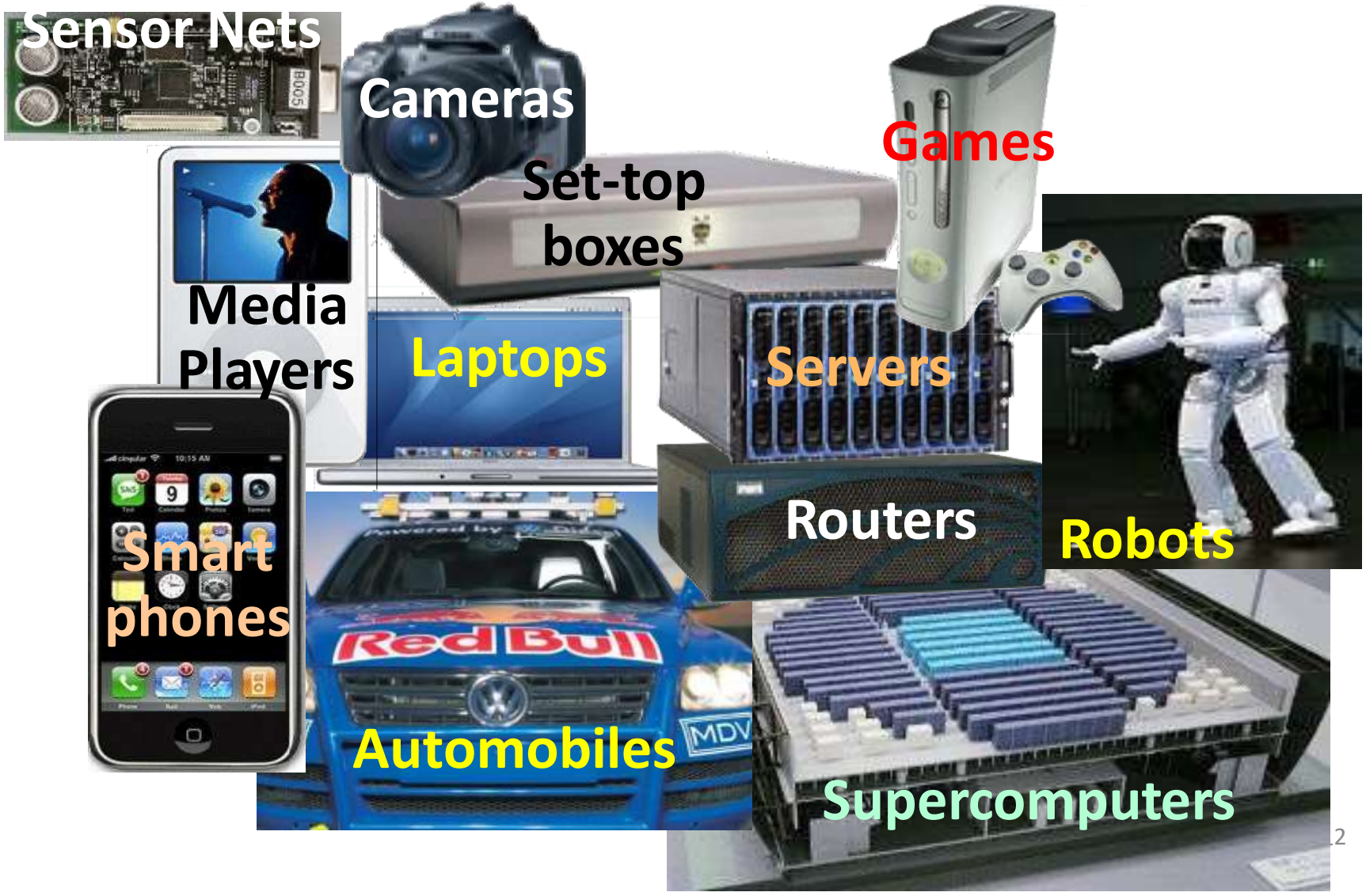
- Restrict what can be done efficiently
- New technologies make new arch possible

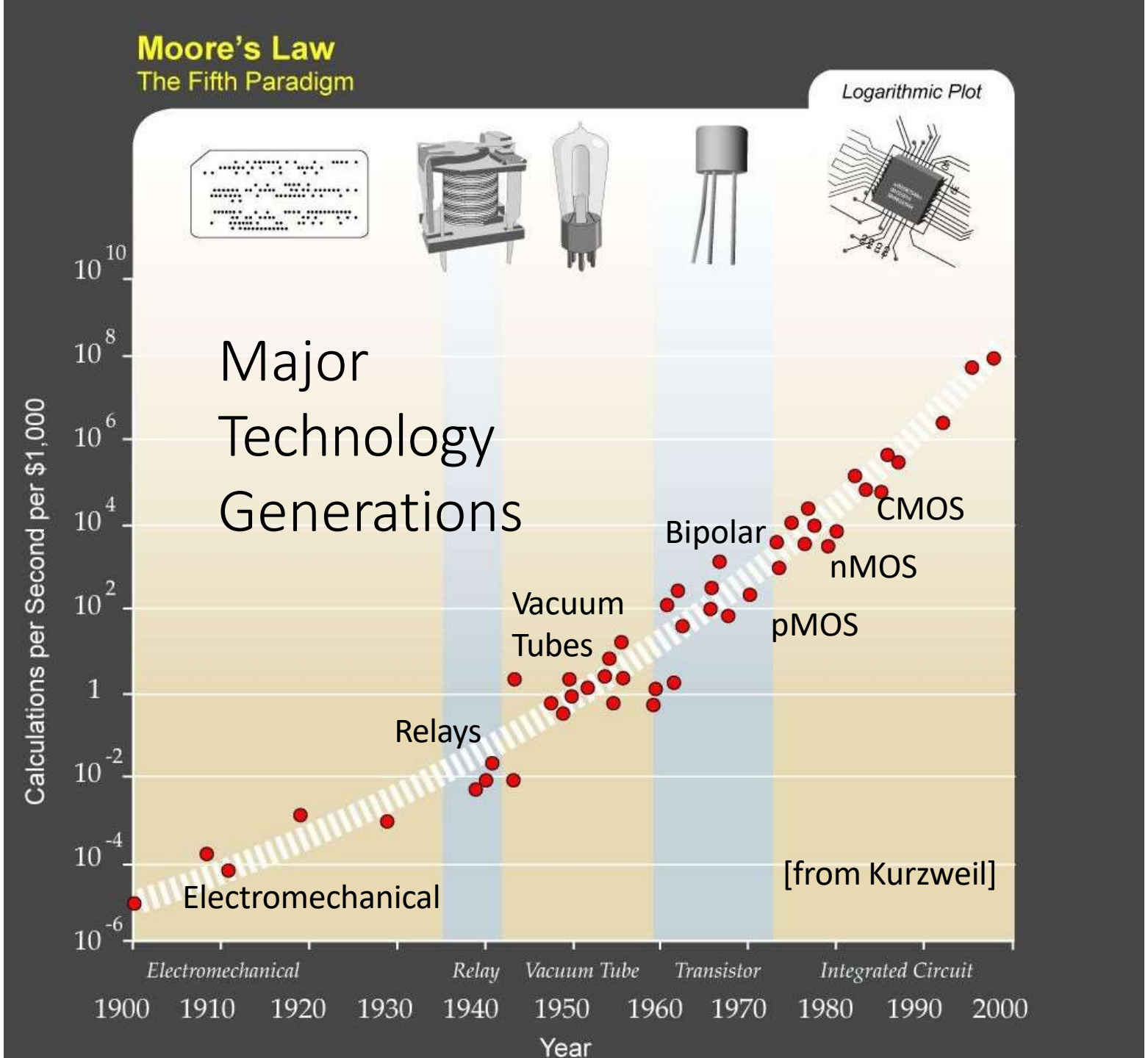
Computers Then…



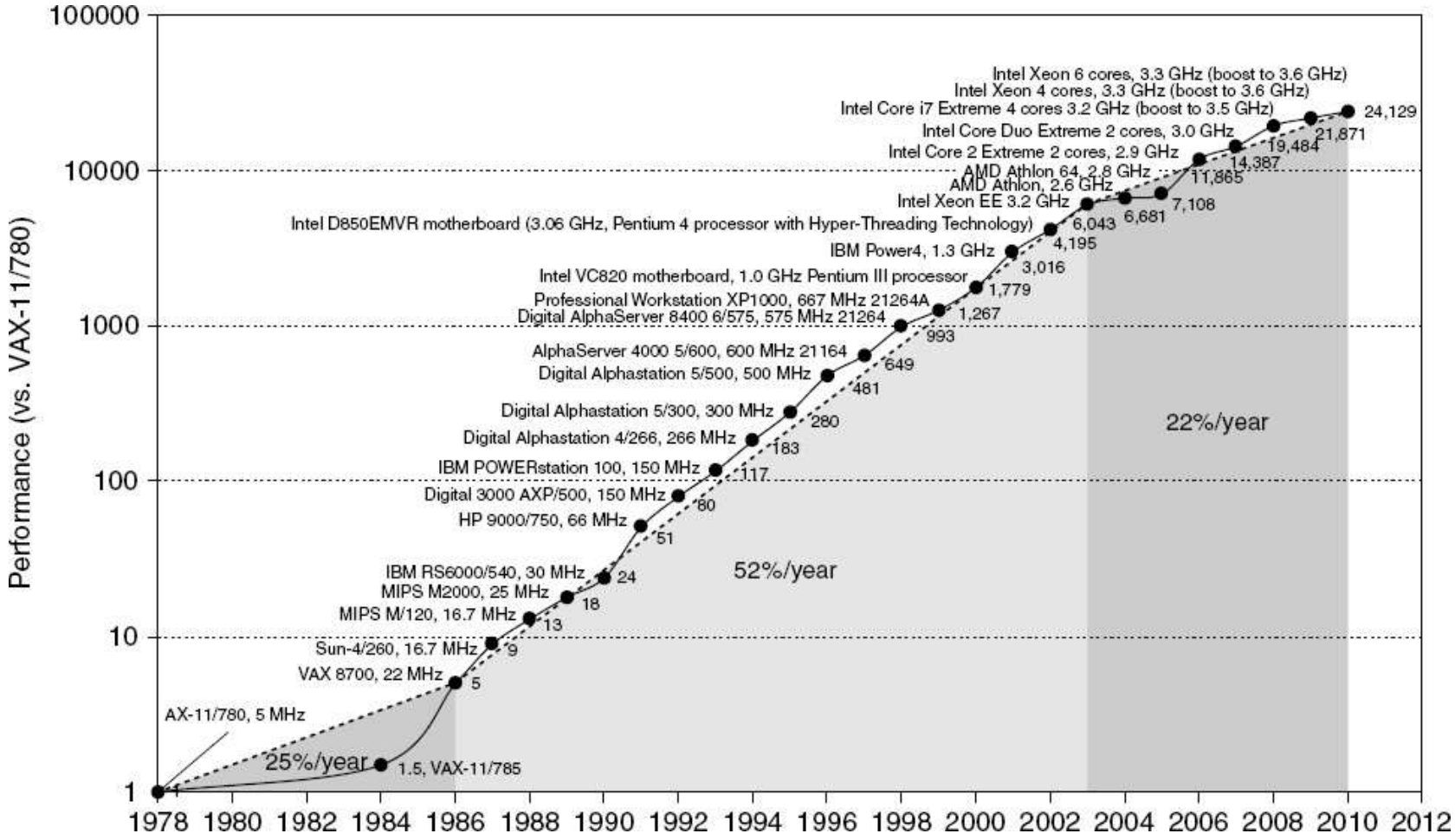
IAS Machine. Design directed by John von Neumann.
First booted in Princeton NJ in 1952
Smithsonian Institution Archives (Smithsonian Image 95-06151)

Computers Now



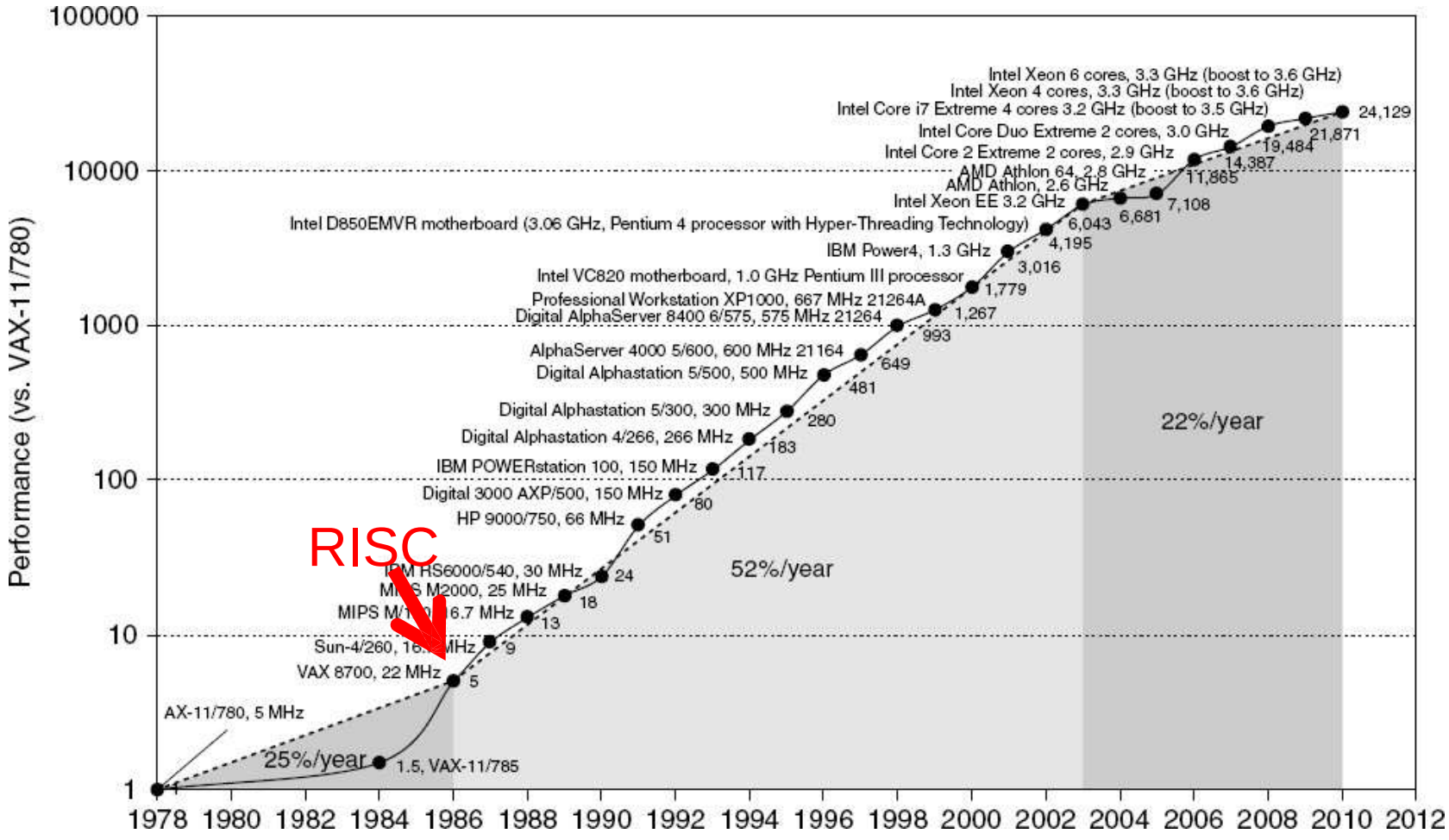

Moore's Law
The Fifth Paradigm
Logarithmic Plot
Major Technology Generations
Calculations per Second per $1,000
10^{10}
10^{8}
10^{6}
10^{4}
10^{2}
1
10^{-2}
10^{-4}
10^{-6}
Electromechanical
Relays
Vacuum Tubes
Bipolar
CMOS
nMOS
pMOS
[from Kurzweil]
Electromechanical
Relay
Vacuum Tube
Transistor
Integrated Circuit
1900
1910
1920
1930
1940
1950
1960
1970
1980
1990
2000
Year

Sequential Processor Performance



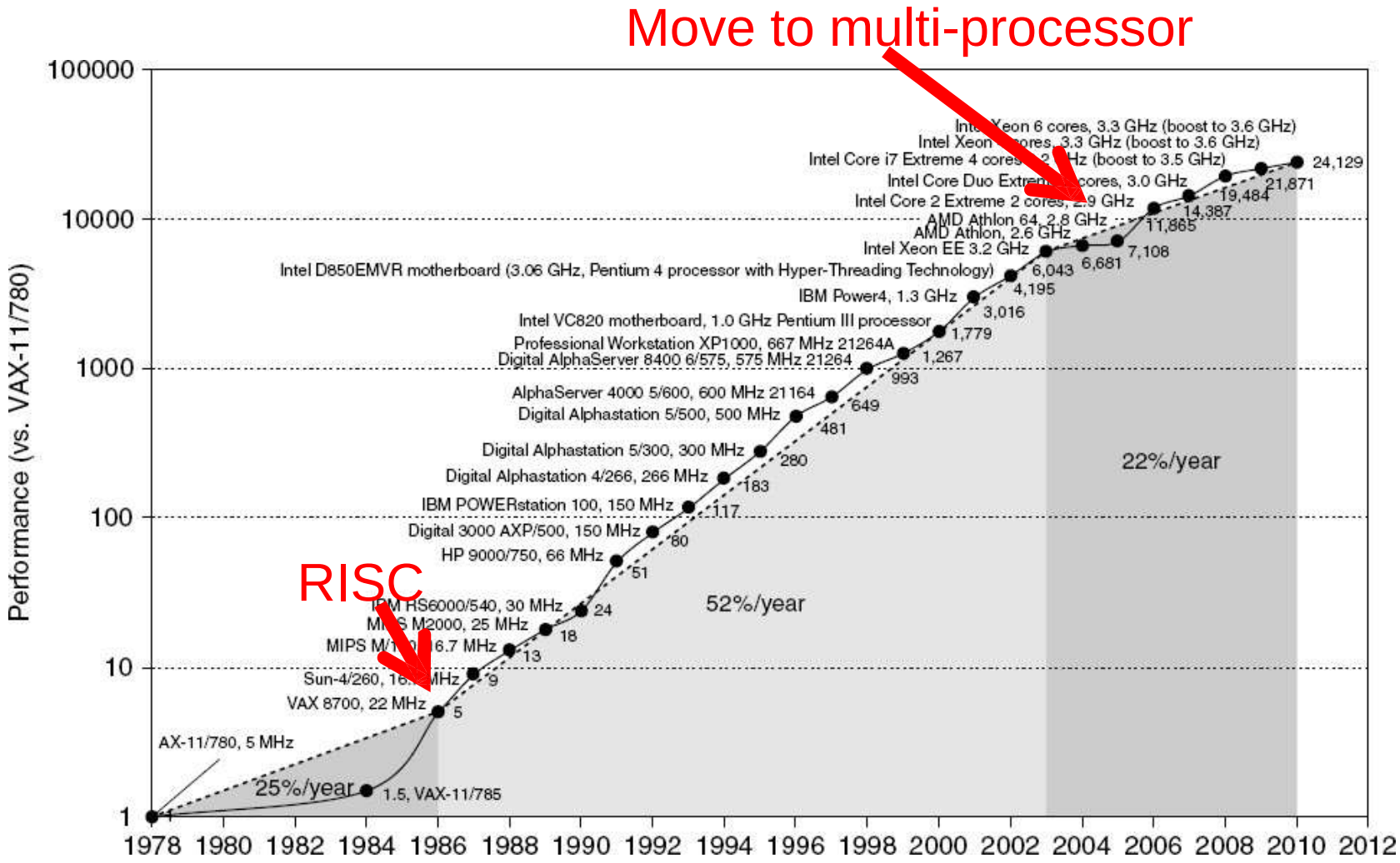
From Hennessy and Patterson Ed. 5 Image Copyright © 2011, Elsevier Inc. All rights Reserved.

Sequential Processor Performance



From Hennessy and Patterson Ed. 5 Image Copyright © 2011, Elsevier Inc. All rights Reserved.

Sequential Processor Performance



From Hennessy and Patterson Ed. 5 Image Copyright © 2011, Elsevier Inc. All rights Reserved.

Course Structure

- Recommended Readings
- In-Lecture Questions
- Problem Sets
  - Very useful for exam preparation
  - Peer Evaluation
- Midterm
- Final Exam

Course Content Computer Organization

Computer Organization

- Basic Pipelined Processor

~50,000 Transistors

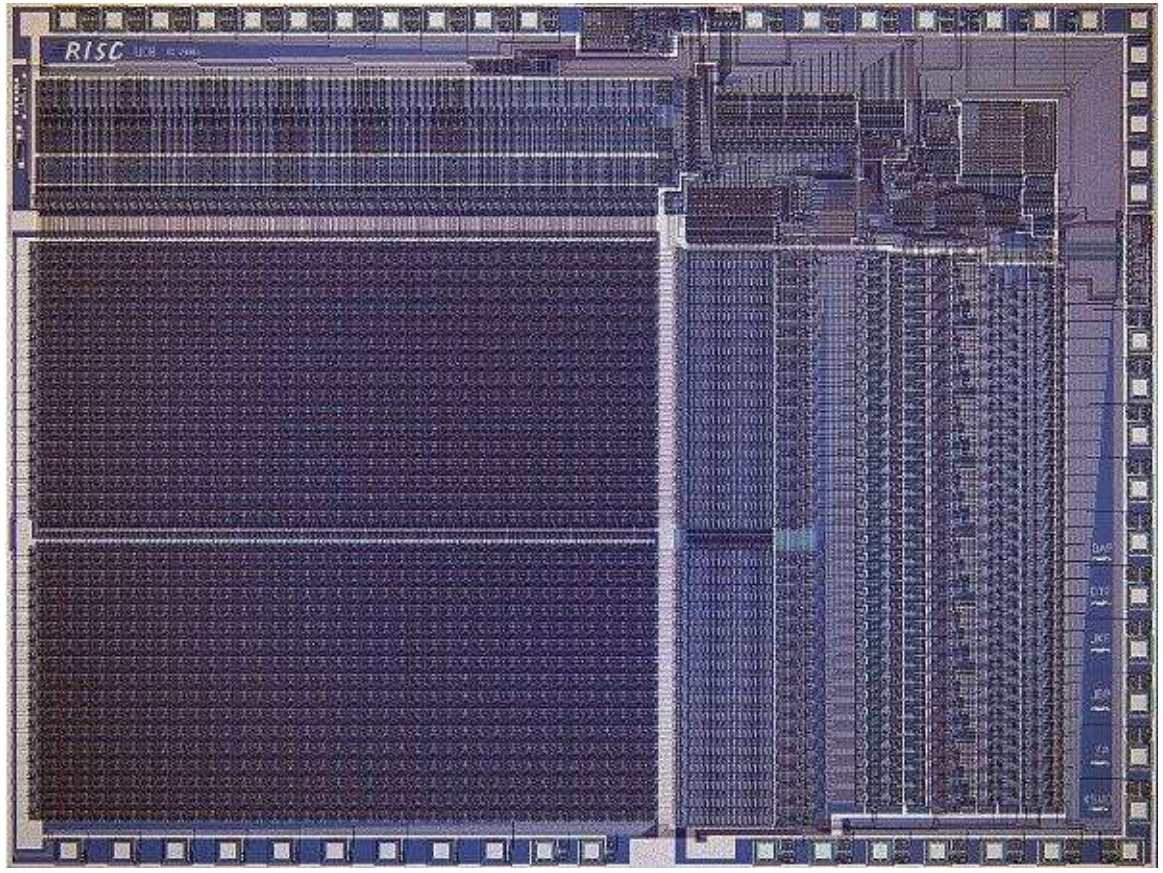


Photo of Berkeley RISC I, © University of California (Berkeley)

Architecture vs. Microarchitecture

"Architecture"/Instruction Set Architecture:

- Programmer visible state (Memory & Register)
- Operations (Instructions and how they work)
- Execution Semantics (interrupts)
- Input/Output
- Data Types/Sizes

Microarchitecture/Organization:

- Tradeoffs on how to implement ISA for some metric (Speed, Energy, Cost)
- Examples: Pipeline depth, number of pipelines, cache size, silicon area, peak power, execution ordering, bus widths, ALU widths

Software Developments

up to 1955 — Libraries of numerical routines

- Floating point operations
- Transcendental functions
- Matrix manipulation, equation solvers, . . .

1955-60 — *High level Languages* - Fortran 1956

Operating Systems -

- Assemblers, Loaders, Linkers, Compilers
- Accounting programs to keep track of usage and charges

Software Developments

up to 1955 Libraries of numerical routines

- Floating point operations
- Transcendental functions
- Matrix manipulation, equation solvers, . . .

1955-60 *High level Languages* - Fortran 1956

Operating Systems -

- Assemblers, Loaders, Linkers, Compilers
- Accounting programs to keep track of usage and charges

Machines required *experienced operators*

- Most users could not be expected to understand these programs

Compatibility Problem at IBM

By early 1960's, IBM had 4 incompatible lines of computers!

701	$\Rightarrow$	7094
650	$\Rightarrow$	7074
702	$\Rightarrow$	7080
1401	$\Rightarrow$	7010

Compatibility Problem at IBM

By early 1960's, IBM had 4 incompatible lines of computers!

701	⇒	7094
650	⇒	7074
702	⇒	7080
1401	⇒	7010

Each system had its own

- Instruction set
- I/O system and Secondary Storage:
  magnetic tapes, drums and disks
- assemblers, compilers, libraries,...
- market niche business, scientific, real time, ...

Compatibility Problem at IBM

By early 1960's, IBM had 4 incompatible lines of computers!

701	⇒	7094
650	⇒	7074
702	⇒	7080
1401	⇒	7010

Each system had its own

- Instruction set
- I/O system and Secondary Storage:
  magnetic tapes, drums and disks
- assemblers, compilers, libraries,...
- market niche business, scientific, real time, ...

⇒ IBM 360

Same Architecture Different Microarchitecture

AMD Phenom X4

- X86 Instruction Set
- Quad Core
- 125W
- Decode 3 Instructions/Cycle/Core
- 64KB L1 I Cache, 64KB L1 D Cache
- 512KB L2 Cache
- Out-of-order
- 2.6GHz

Intel Atom

- X86 Instruction Set
- Single Core
- 2W
- Decode 2 Instructions/Cycle/Core
- 32KB L1 I Cache, 24KB L1 D Cache
- 512KB L2 Cache
- In-order
- 1.6GHz

Different Architecture Different Microarchitecture

AMD Phenom X4

- X86 Instruction Set
- Quad Core
- 125W
- Decode 3 Instructions/Cycle/Core
- 64KB L1 I Cache, 64KB L1 D Cache
- 512KB L2 Cache
- Out-of-order
- 2.6GHz

IBM POWER7

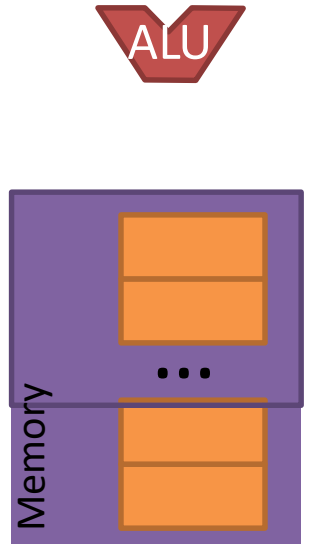
- Power Instruction Set
- Eight Core
- 200W
- Decode 6 Instructions/Cycle/Core
- 32KB L1 I Cache, 32KB L1 D Cache
- 256KB L2 Cache
- Out-of-order
- 4.25GHz

Where Do Operands Come from And Where Do Results Go?

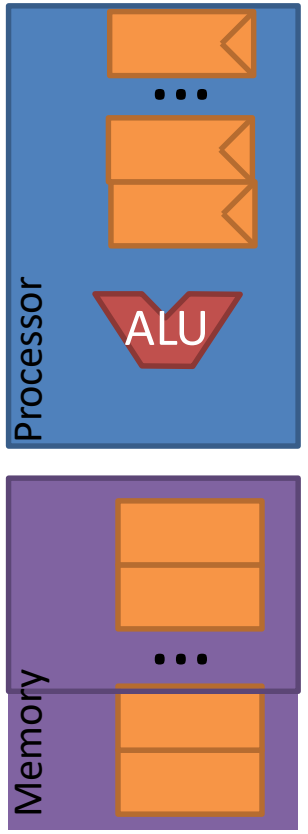
Where Do Operands Come from And Where Do Results Go?



Where Do Operands Come from And Where Do Results Go?



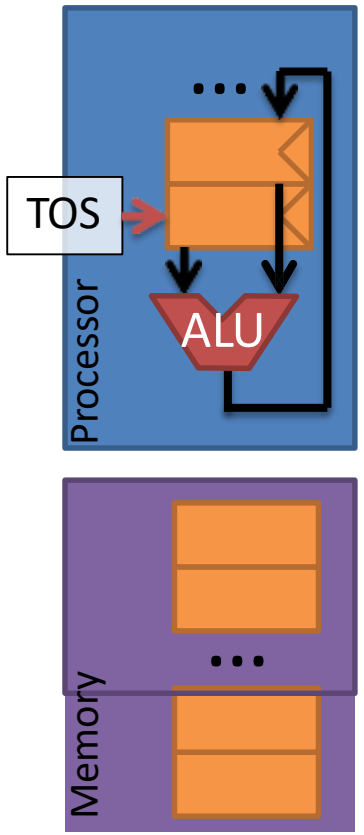
Where Do Operands Come from And Where Do Results Go?



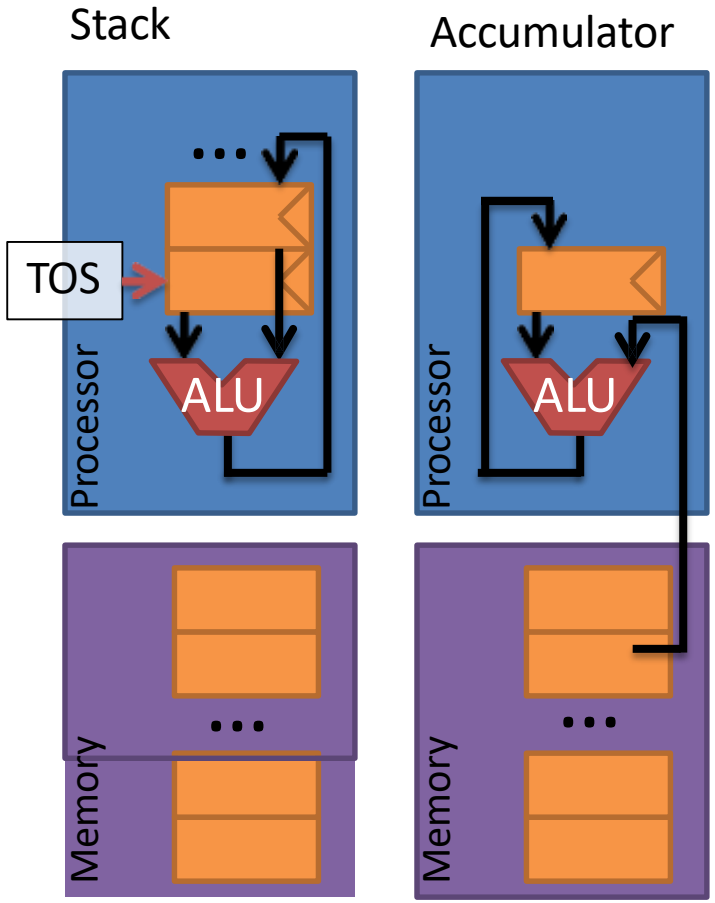
Where Do Operands Come from And Where Do Results Go?

Where Do Operands Come from And Where Do Results Go?

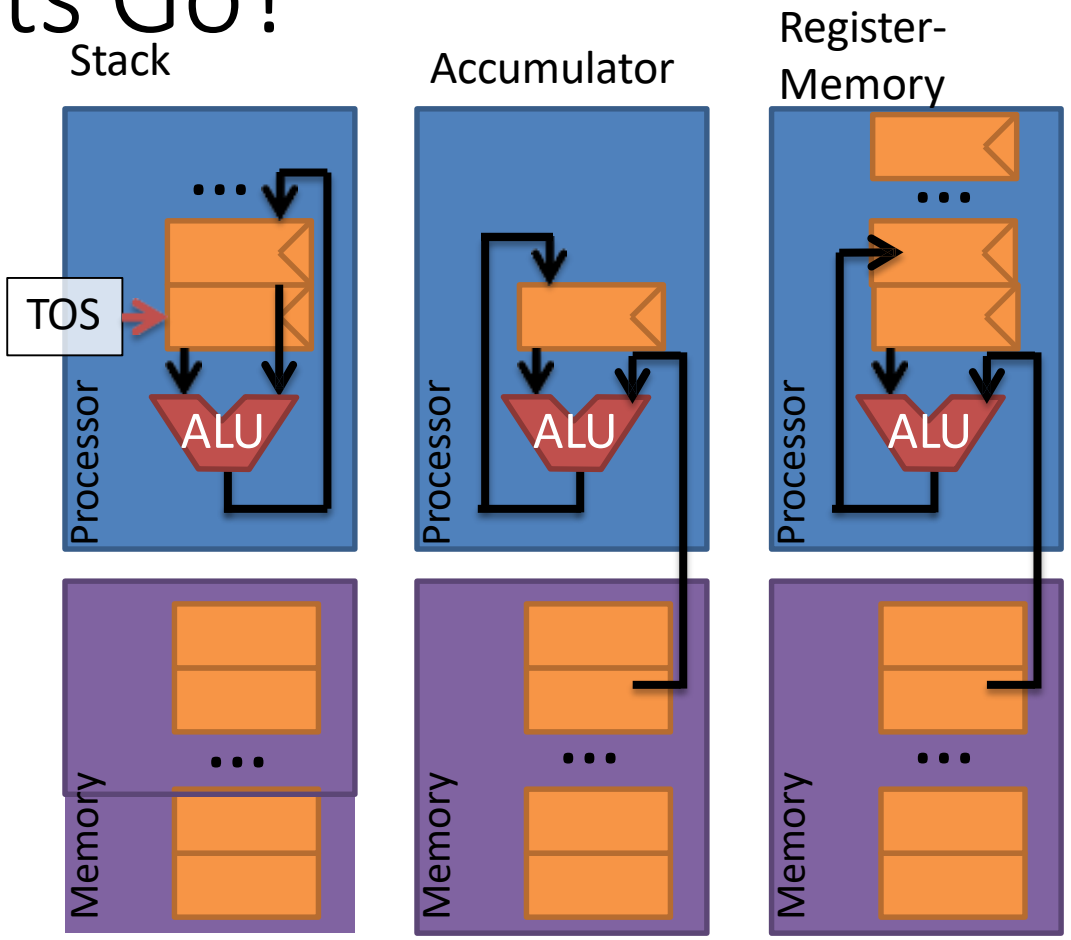
Stack



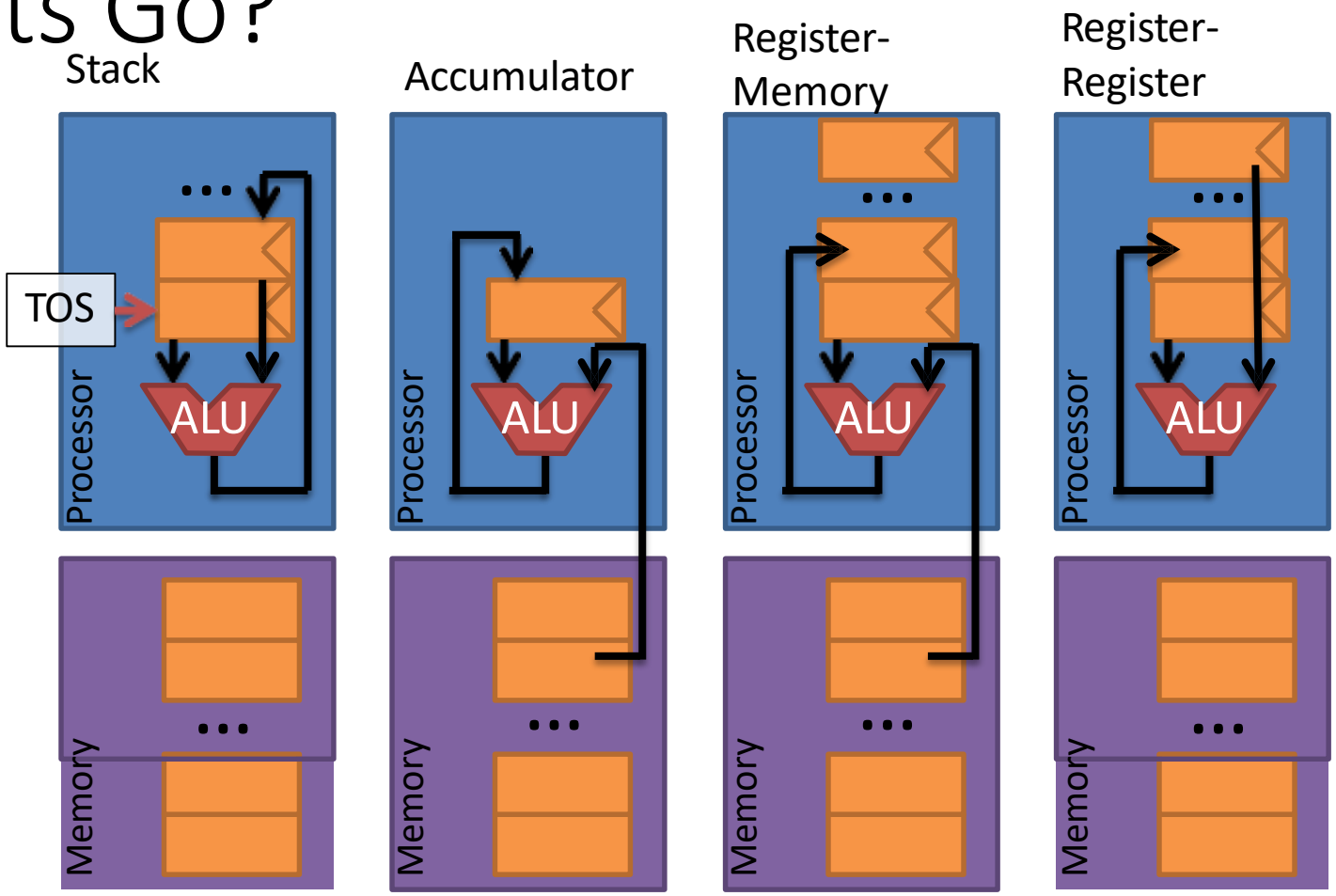
Where Do Operands Come from And Where Do Results Go?



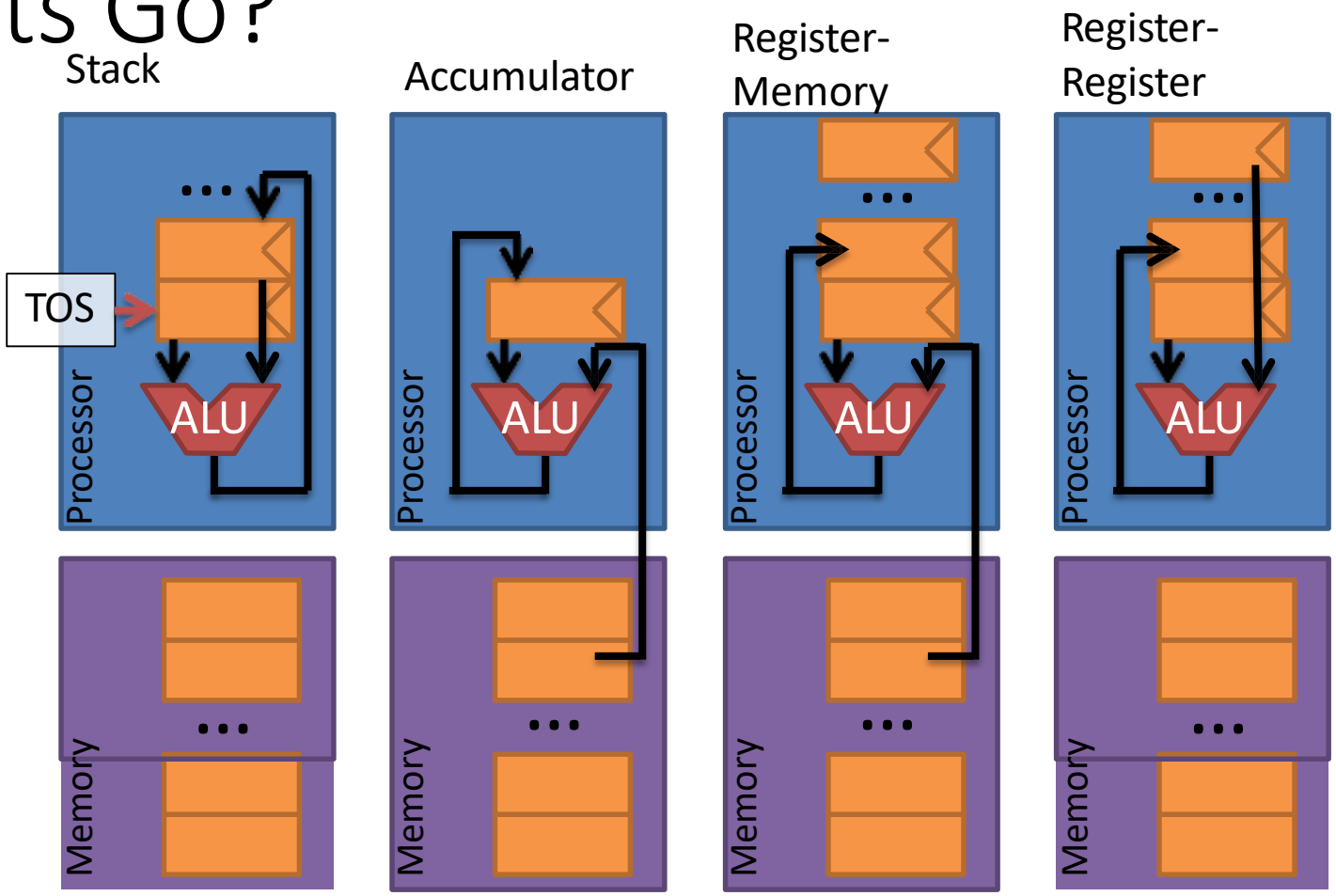
Where Do Operands Come from And Where Do Results Go?



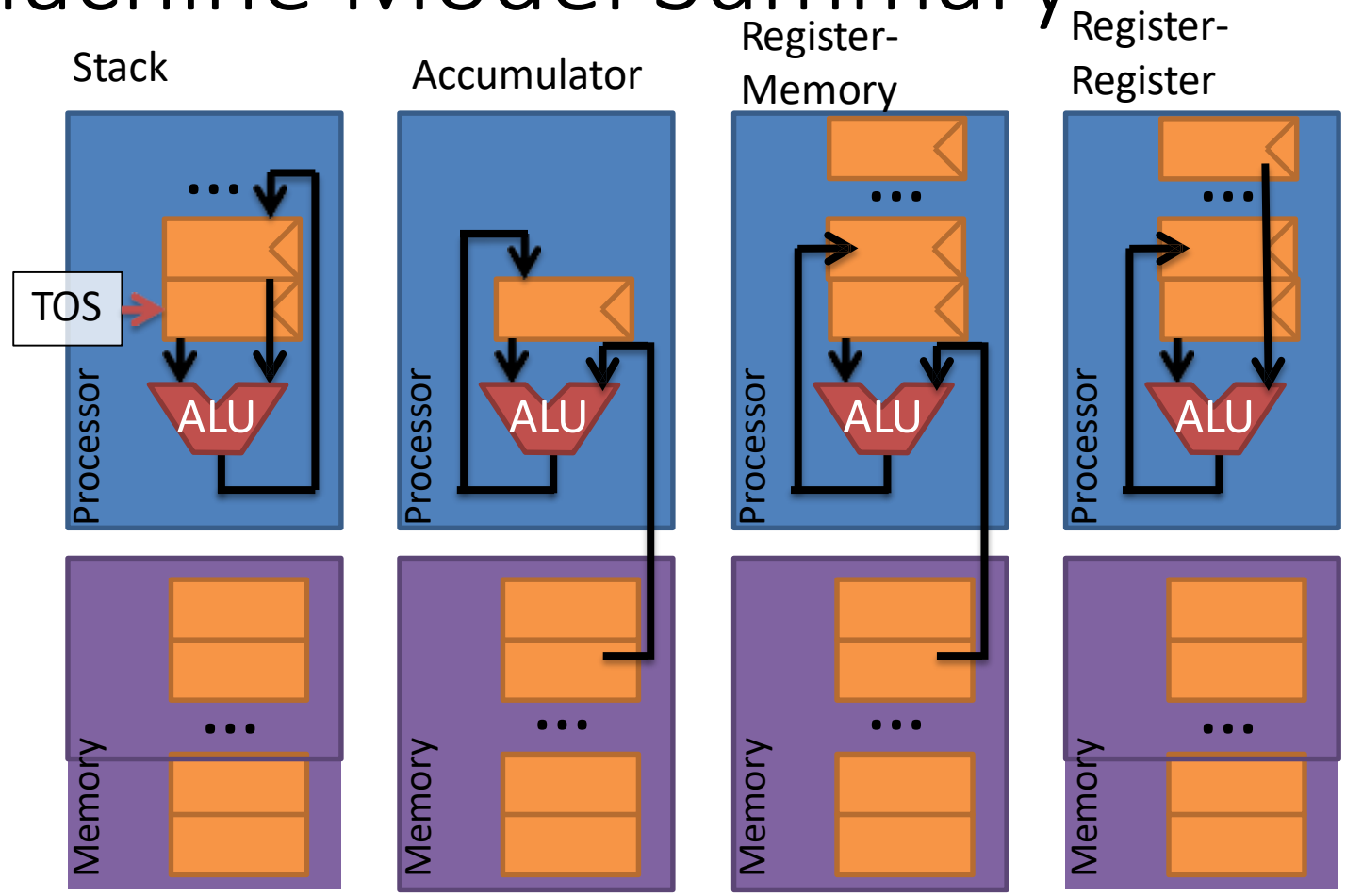
Where Do Operands Come from And Where Do Results Go?



Where Do Operands Come from And Where Do Results Go?

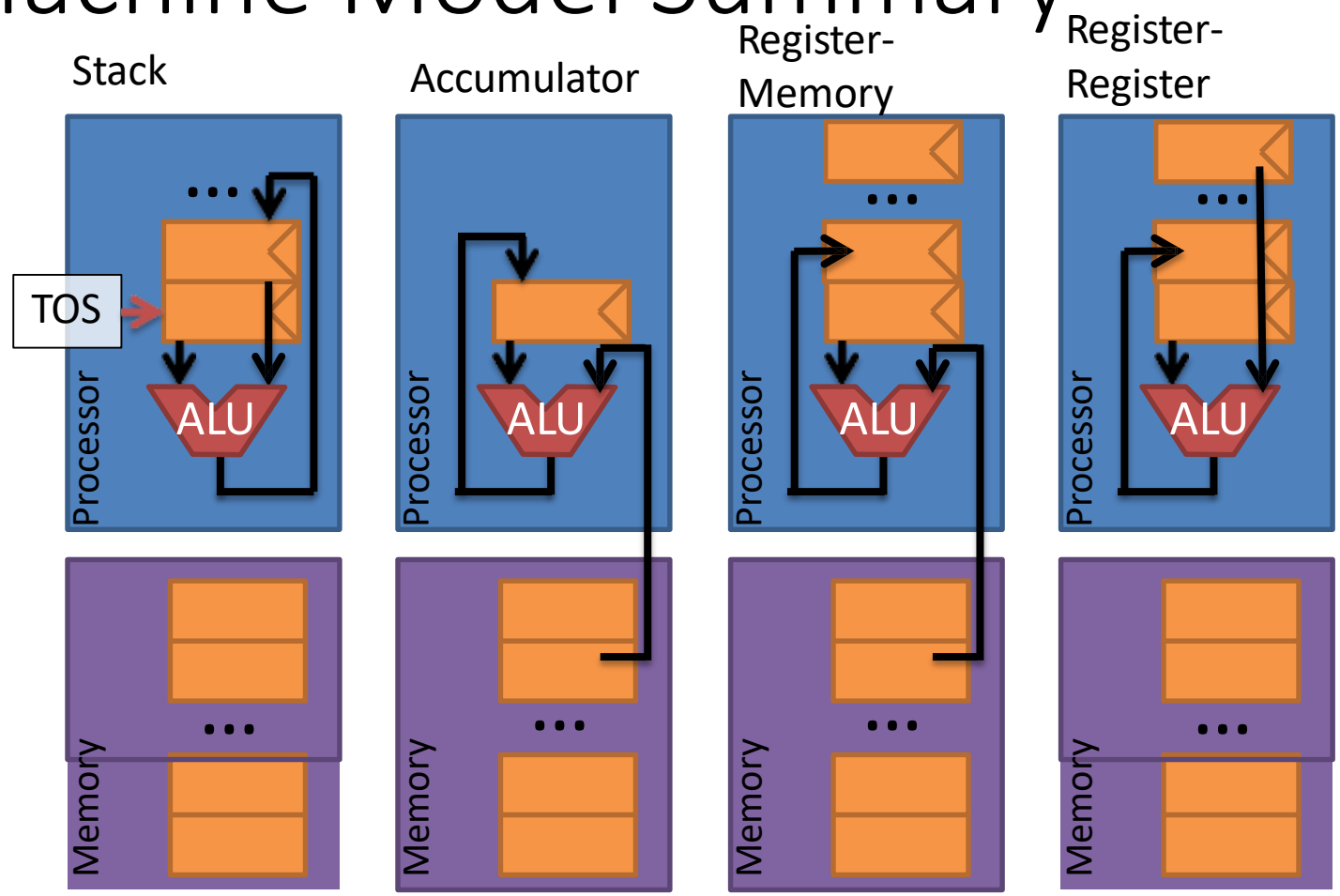


Machine Model Summary



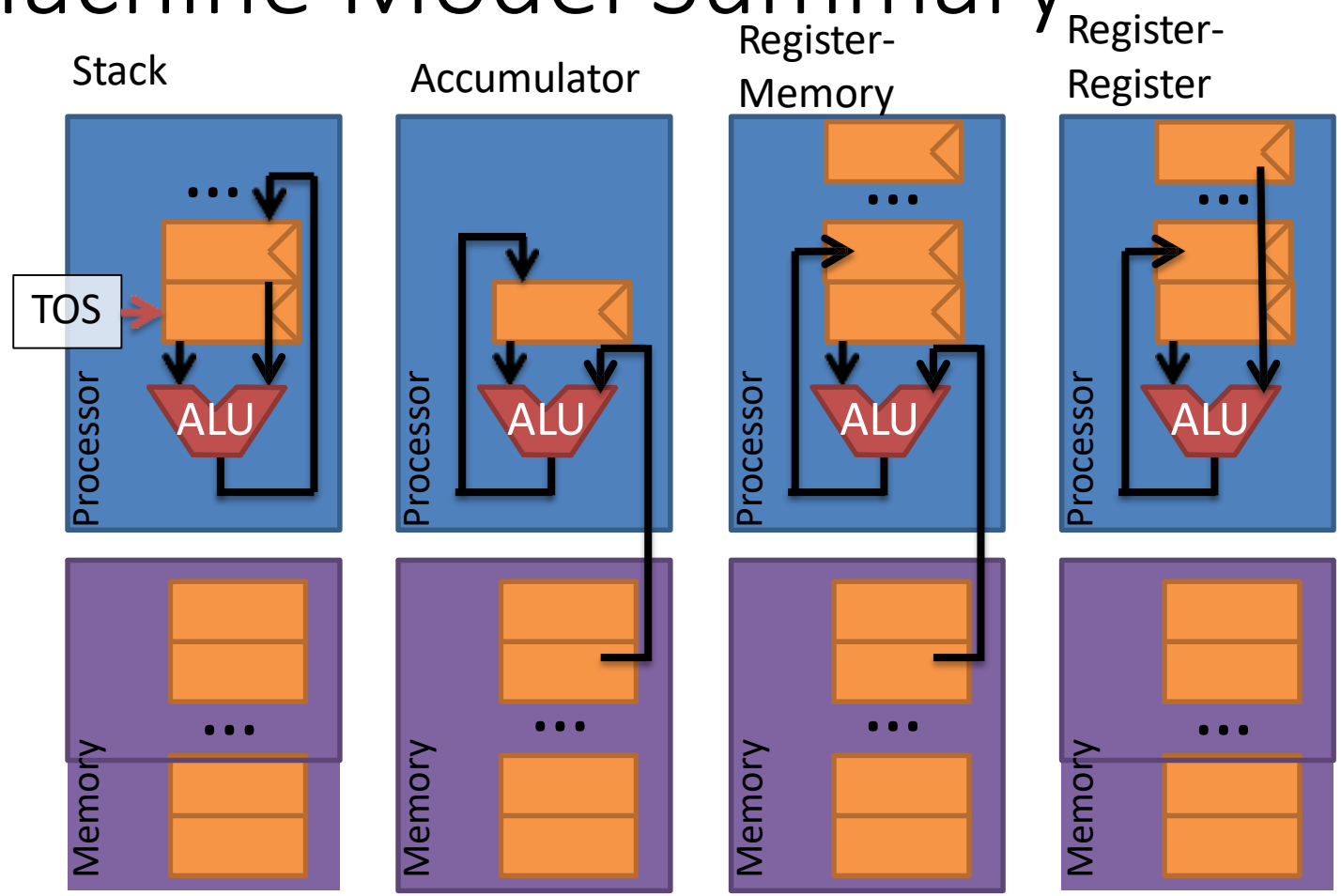
Machine Model Summary

C = A + B



Machine Model Summary

C = A + B



Stack	Accumulator	Register-Memory	Register-Register
Push A	Load A	Load R1, A	Load R1, A
Push B	Add B	Add R3, R1, B	Load R2, B
Add	Store C	Store R3, C	Add R3, R1, R2
Pop C			Store R3, C

Real World Instruction Sets

Arch	Type	# Oper	# Mem	Data Size	# Regs	Addr Size	Use
Alpha	Reg-Reg	3	0	64-bit	32	64-bit	Workstation
ARM	Reg-Reg	3	0	32/64-bit	16	32/64-bit	Cell Phones, Embedded
MIPS	Reg-Reg	3	0	32/64-bit	32	32/64-bit	Workstation, Embedded
SPARC	Reg-Reg	3	0	32/64-bit	24-32	32/64-bit	Workstation
TI C6000	Reg-Reg	3	0	32-bit	32	32-bit	DSP
IBM 360	Reg-Mem	2	1	32-bit	16	24/31/64	Mainframe
x86	Reg-Mem	2	1	8/16/32/ 64-bit	4/8/24	16/32/64	Personal Computers
VAX	Mem-Mem	3	3	32-bit	16	32-bit	Minicomputer
Mot. 6800	Accum.	1	1/2	8-bit	0	16-bit	Microcontroler

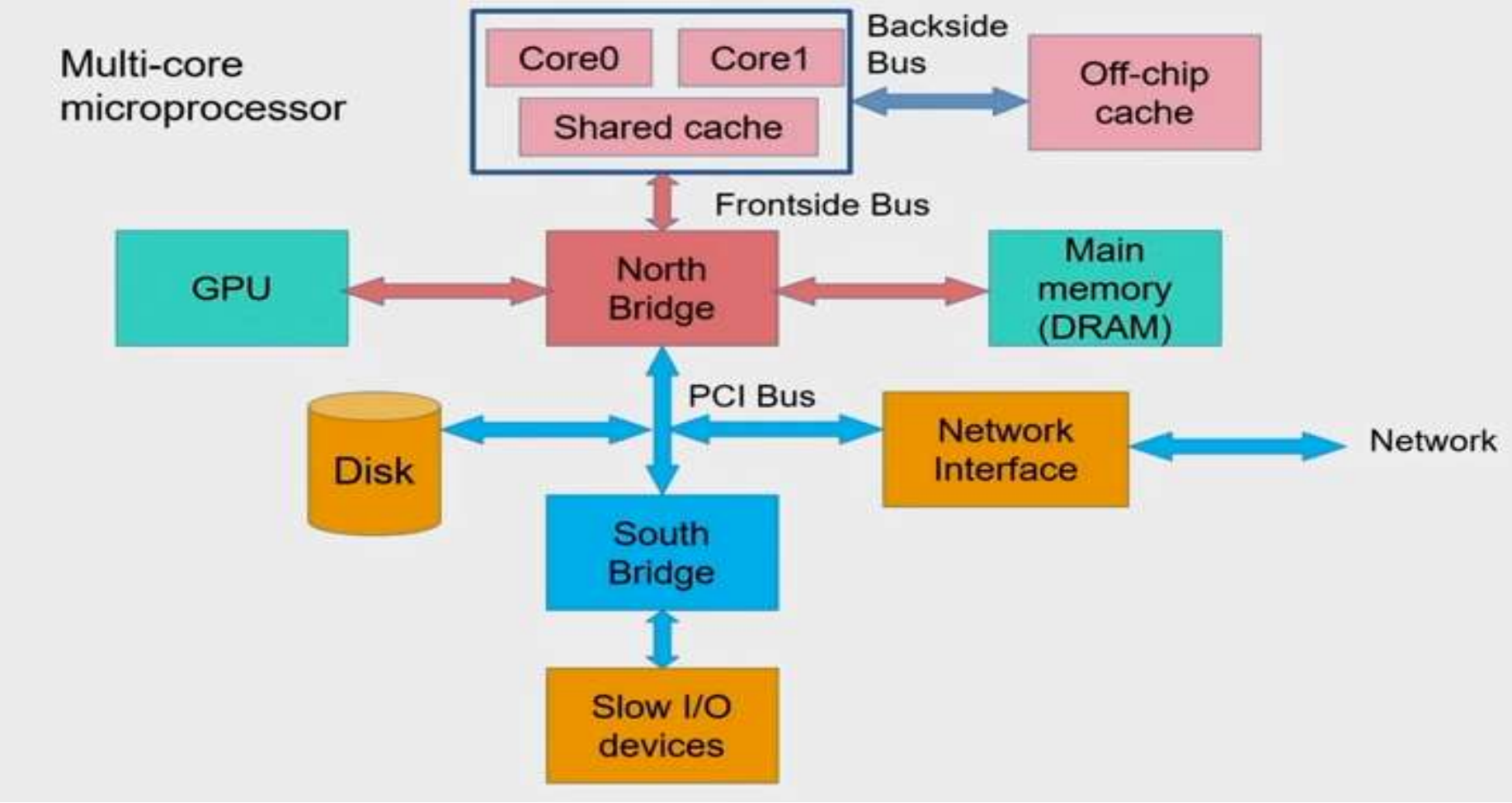
Why we are building Parallel Systems?

- Here we will see how a basic computer system looks like and how do we parallelize it to make a big parallel system
- Single Processor Performance
  - Transistor density
  - Speed
  - ILP
- Speed increases, power increases, heat increases
- How do we exploit the increasing number of transistors without increasing the clock frequency and still get the better performance?
  - Add parallelism
  - Multicore

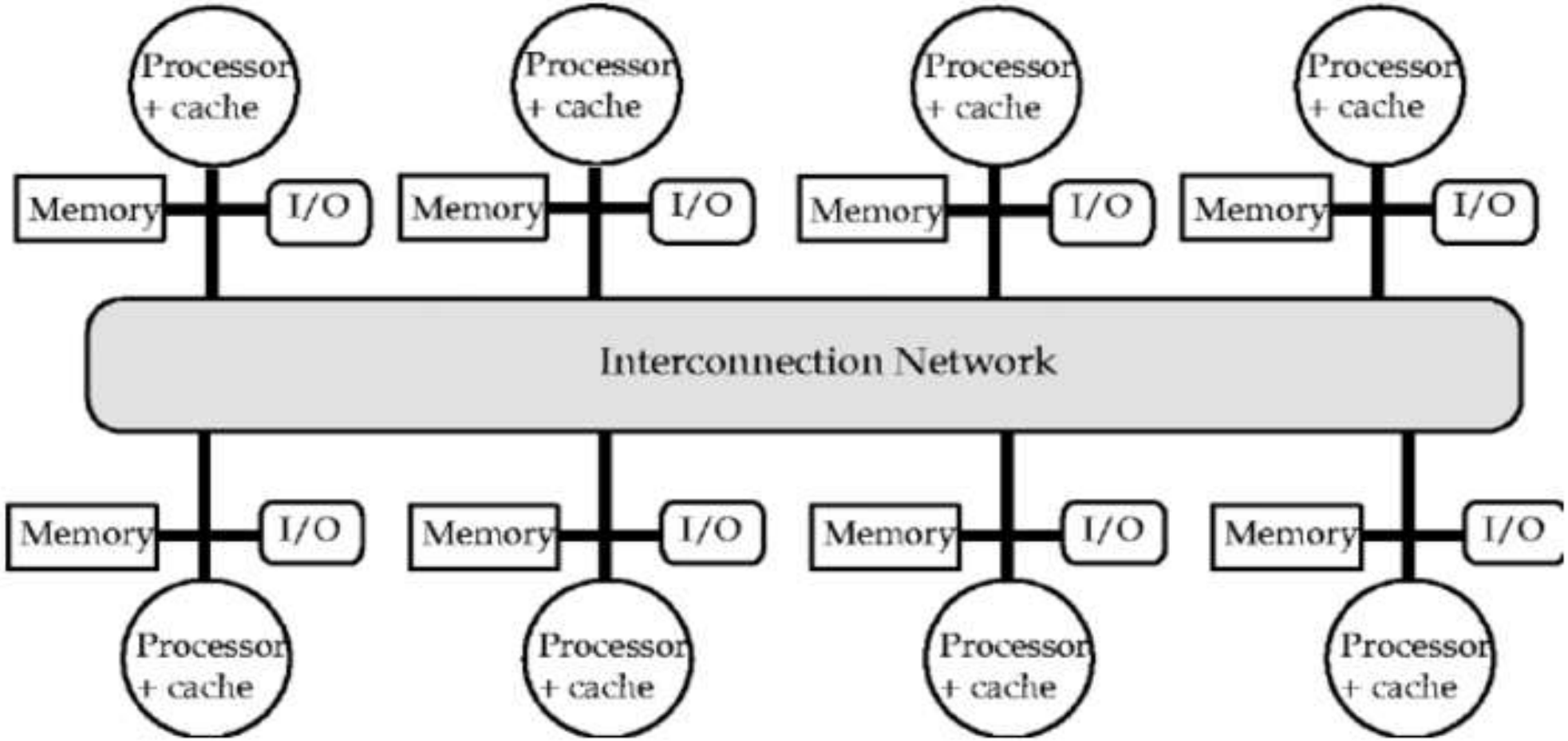
Components of Computer System

- North Bridge Chip
  - The Northbridge part of the chipset controls the high-speed channels
  - The northbridge handles the high-speed communication between the CPU, memory, and graphics card
- PCI chip
  - Peripheral Component Interconnect (PCI) is a standard interface that connects hardware devices to a computer's processor bus.
  - PCI is a high-speed bus that's found in almost all desktop computers
- South Bridge Chip
  - Southbridge controls the lower speed devices.
  - The southbridge manages the slower input/output (I/O) operations and connects devices like hard drives (HD), universal serial bus (USB) devices, and audio interfaces

Components of Computer System



Generic multiprocessor system with distributed memory



Generic Parallel Architecture

- Processor node
- Several processor nodes are connected through global interconnect
  - network helps to transmit data between the nodes
  - Either bus based or point to point
  - Connected using network interface
- Input Output (I/O) devices like hard disk are connected to the global interconnect

Parallelism in Architectures

- Scalar Processor
  - ADD R1, R2, R3
  - Fetch, Decode, Execute
- ILP – Instruction level Parallelism
- TLP – Thread Level Parallelism
- Vector and Array Processor

Instruction Level Parallelism

- Instruction-level parallelism (ILP) is a technique that allows a computer processor to execute multiple instructions at the same time.
- If the instructions are independent then we can exploit this parallelism
- If the instruction are dependent then we cannot exploit this parallelism because the result of one will be required by the result of another

ILP – vector addition example

- For(j=0; j<1000; j++)
  - A[j] = B[j] + C[j]
- Here each loop iteration is concurrent and will run parallelly, implies we have 1000 possible instructions which can be done in parallel
- Processor will take help from the compiler to do this because as a programmer we have written that for loop and we don't try to understand where is the ILP or help the processor exploit ILP.
- WHO HELPS to exploit ILP?
  - It's the job of compiler and the micro architecture

Thread Level Parallelism

- Splits code in to parallel threads which can execute independently on multiple core
- In fact threads may have to synchronize with each other on some occasions to exchange data values
- TLP is the task of the programmer
  - Need to design new algorithms
  - Re-structure code to expose and manage threads
    - coordinating and integrating multiple channels and technologies to improve communication and customer experience
    - Synchronization among threads – it is a process that ensures that multiple threads work together to reach a known point of operation before continuing.

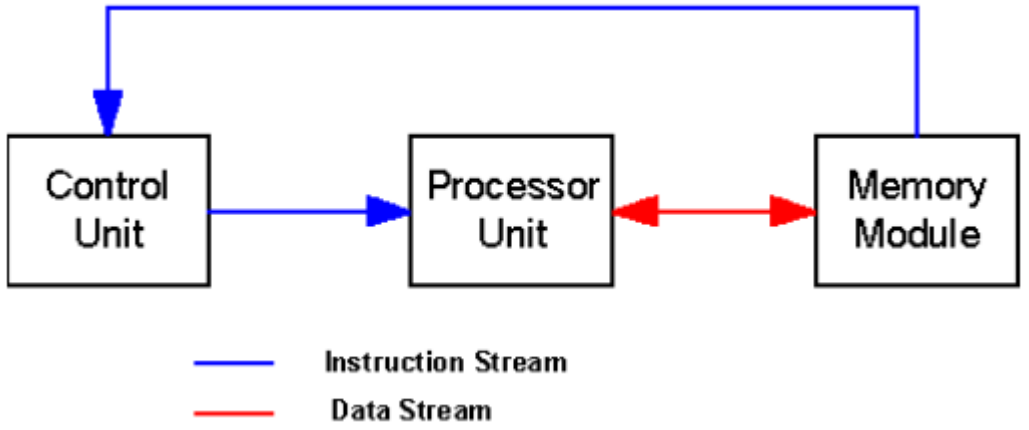
Parallel Architecture Classification Scheme

- There are three computer architectural classification scheme based on the multiplicity of instructions and data streams. They are as given below:

1) Flynn's Classification'(1966) – SISD, SIMD, MISD, MIMD
2) Feng's Classification (1972)
3) Handler's Classification (1977)

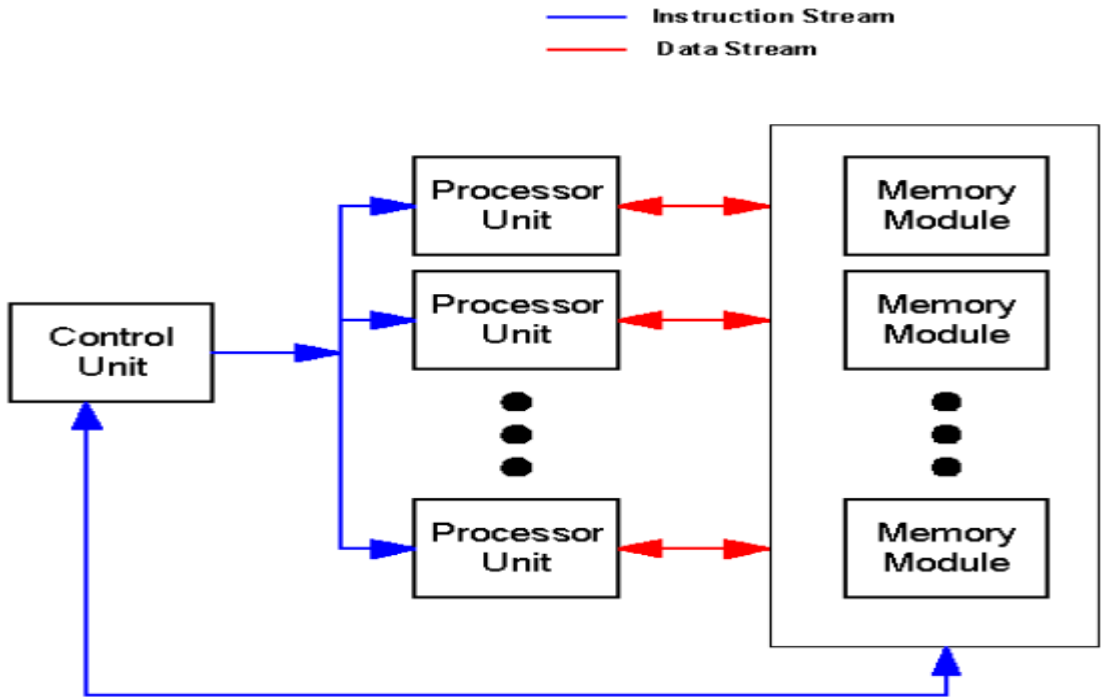
Flynn's Classification – SISD computer

- Single instruction stream single data stream
- One functional unit
  - IBM 701, IBM 1620
- Multiple functional unit
  - IBM 360-91, CDC 660



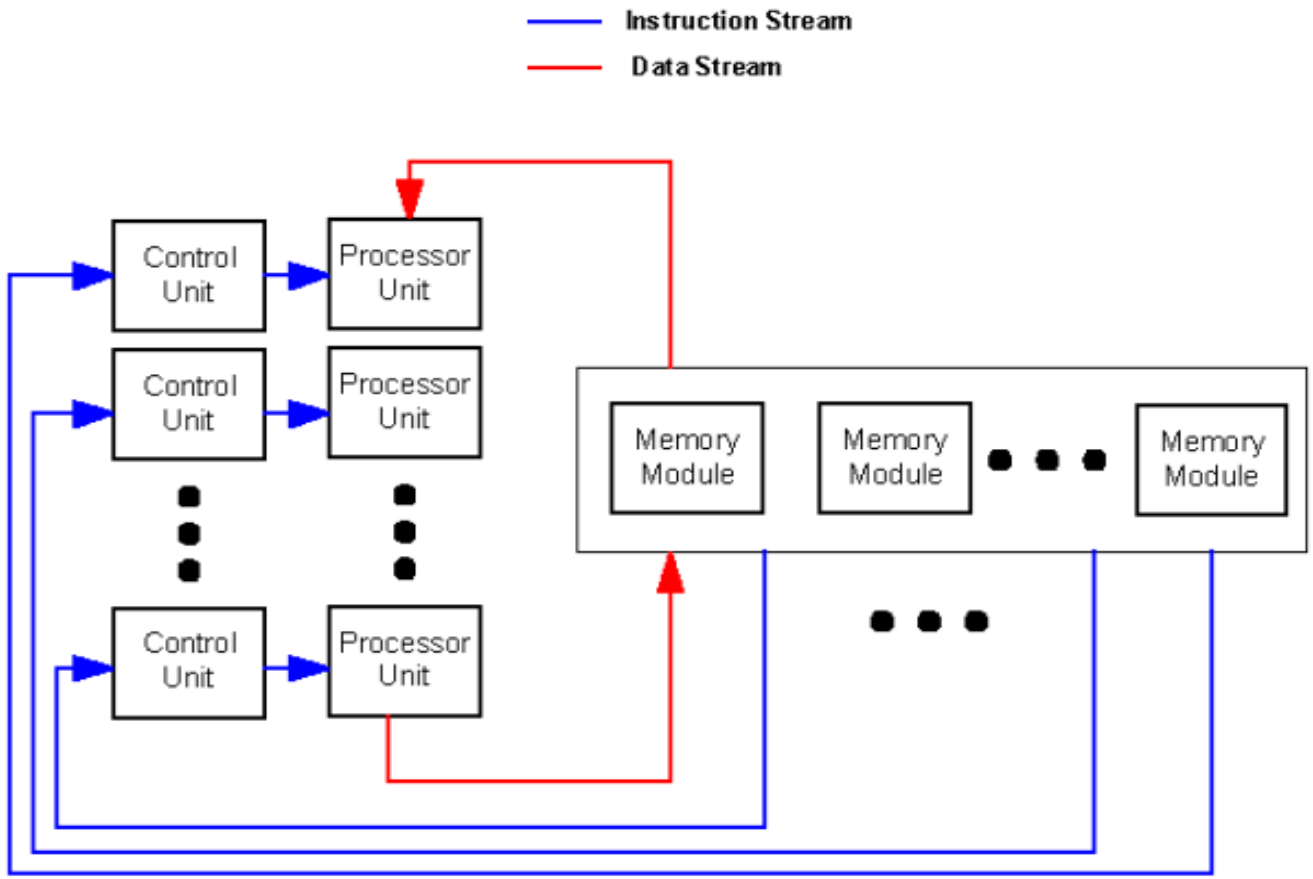
Flynn's Classification – SIMD computer

- Single instruction stream multiple data stream
- Example
  - Illiac IV
  - STARAN



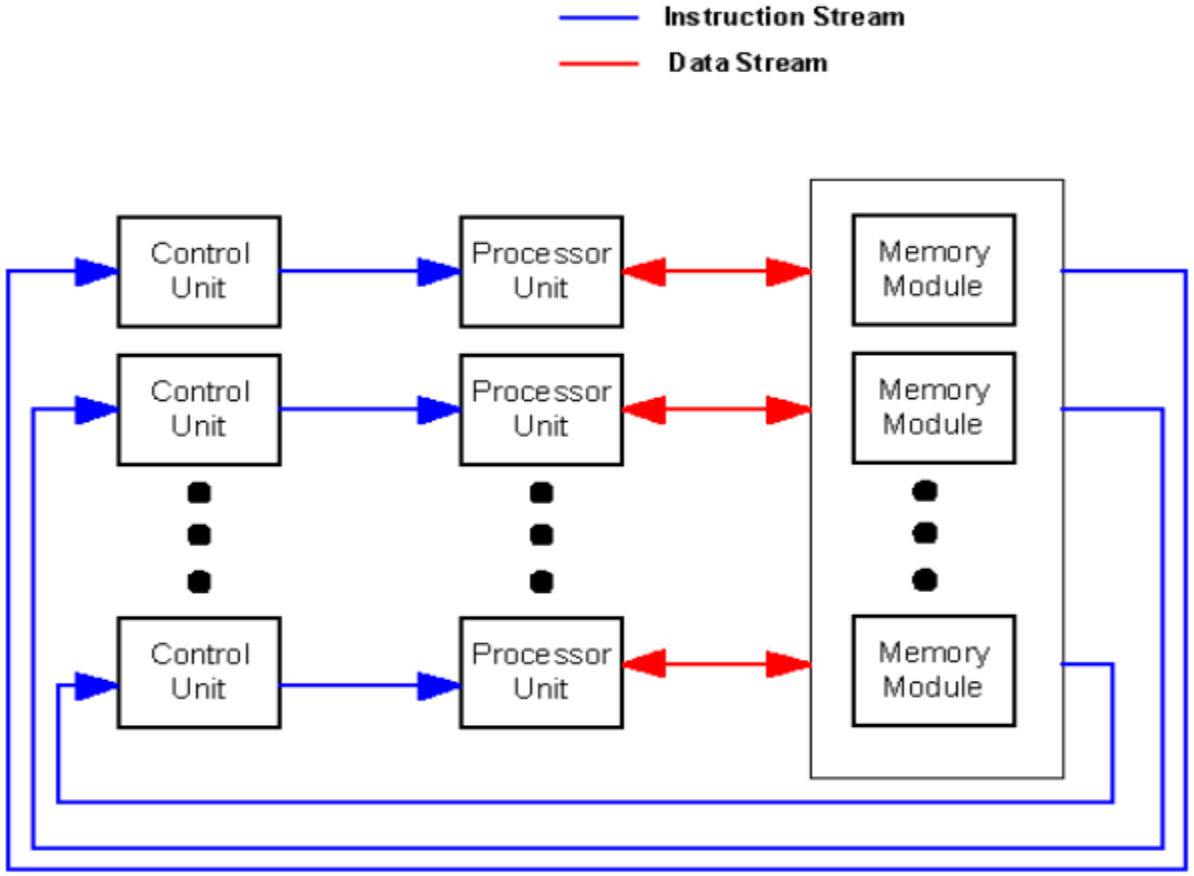
Flynn's Classification – MISD computer

- Multiple instruction stream single data stream
- Here different processing elements run different programs on the same data path



Flynn's Classification – MIMD computer

- Multiple instruction stream Multiple data stream
- Example
  - UNIVAC 1100
  - Borroughs D-825
- Tightly coupled MIMD
- Loosely coupled MIMD
  - Multiple SISD



Feng’s Classification

- Feng’s Classification was given by Feng in the year 1972.
- This is based on degree of Parallelism (DOP).
- It is defined as number of bits that can be processed in unit time is called Degree of Parallelism.

Feng's Classification

Utilisation rate (μ) is given by the following formula:

$$\mu = \frac{\text{Average Degree of Parallelism } (P_a)}{\text{Maximum Degree of Parallelism } (P)}$$

$$\mu = \frac{P_a}{P}$$

If $\mu = 1$ or 100 % then $P_a = P$

That is computing power of all processors is fully utilized.

Average Parallelism (Pa) is given by the formula:

$$P_a = \frac{1}{T} \sum_{i=1}^{T} (P_i)$$

Where, P_i is the maximum number of bits that can be processed in i^{th} cycle, and T is the number of cycles.

i.e. P_1 means P_1 number of bits can be processed in 1st cycle .

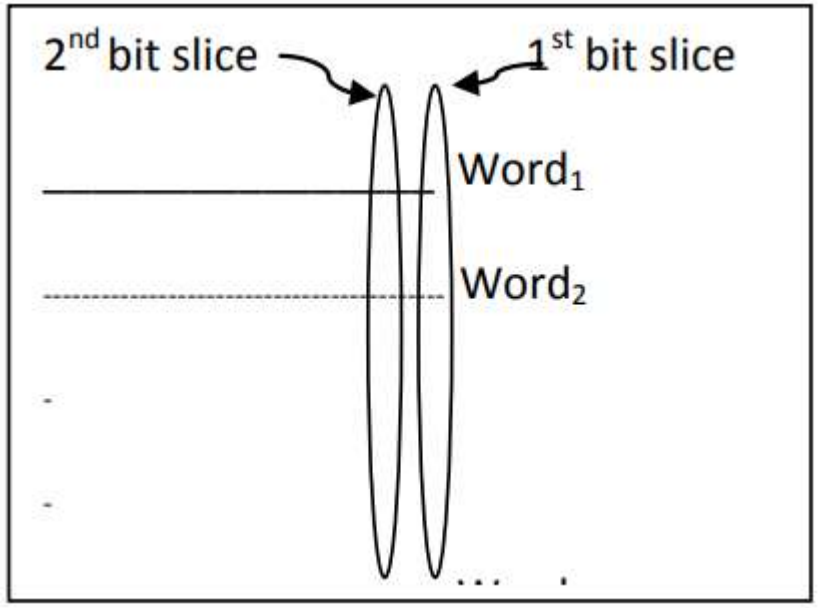
P_2 means P_2 number of bits can be processed in 2nd cycle etc.

Feng's Classification

- Maximum degree of parallelism P of a Computer C, denoted as P(C) is given by P(C) =m.n

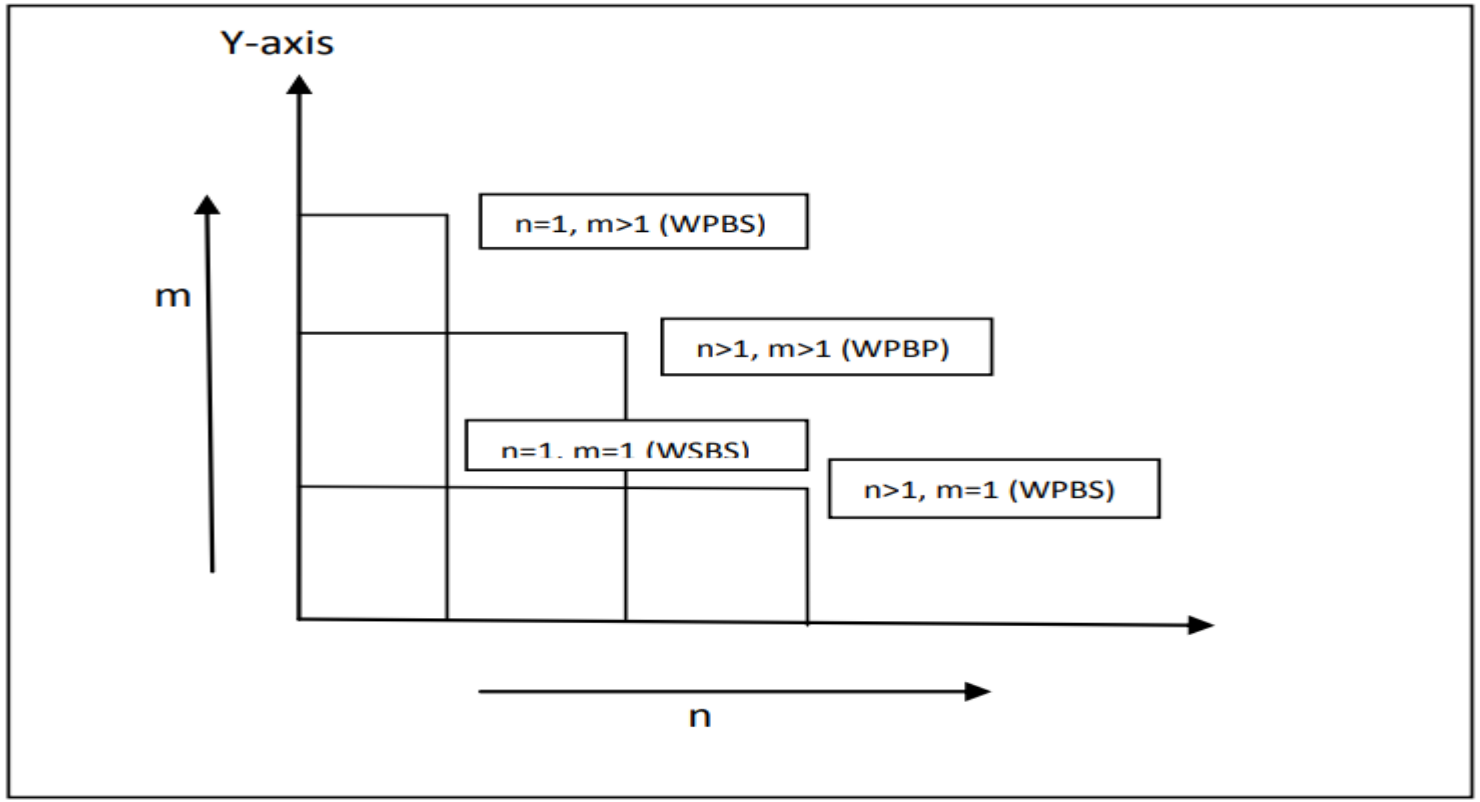
  where, m = Bit slice length and n = Word length

- Bit slice is string of bits taken from all words of same vertical positions.



Feng's Classification

- If m is on Y-axis, and n is on X-axis, then
- Then, mn shows area or maximum degree of parallelism.



Feng's Classification

- Accordingly Computers can be classified as
- WSBS (Word Serial Bit Serial)
- WSBP (Word Serial Bit Parallel)
- WPBS (Word Parallel Bit Serial)
- WPBP (Word Parallel Bit Parallel)

Feng's Classification

- WSBS is a slow process. WSBS was done in 1st generation Computer.
- In WSBS 1 bit is processed at a time.
- WPBP provides highest degree of parallelism that is fastest Computer.

  Example:
  - WSBS – MINIMA (1, 1)
  - WSBP - Burrough 7700 (48, 1)
  - WPBS – DAP (1, 4096)
  - WPBP - TI-ASC (64,32)

  "TI-ASC has a word length of 64 bits and there are 4 arithmetic pipelines in it, each pipeline consisting of 8 stages. Thus, 8 x 4 =32 bits are processed together at one time. Therefore, TI-ASC will be represented on graph as (64,32)"

Handler’s Classification

- In 1977, Wolfgang Handler presented a computer architectural classification scheme for determining the degree of parallelism and pipelining built into the computer system hardware.
- In Handler’s classification pipeline processing systems are divided into three subsystems:
  - **Processor Control Unit (PCU**): Each PCU corresponds to one processor or one CPU.
  - **Arithmetic Logic Unit (ALU):** ALU is equivalent to the processing element (PE).
  - **Bit Level Circuit (BLC):** BLC corresponds to the combinational logic circuit required for 1-bit operations in ALU.

Handler’s Classification

- Handler’s classification uses three pairs of integers containing 6 independent entities that describe the computer system:

Computer = < K*K’, D*D’, W*W’>

where **K** = number of processors (PCUs) within the computer

K’ = number of PCUs that can be pipelined

D = number of ALUs (PEs) under the control of PCU

D‘ = number of PEs that can be pipelined

W = word length of a PE

W’ = number of pipeline stages in all PEs

Handler's Classification: Example

- Example 1: Let us consider an example of Texas Instruments' Advanced Scientific Computer (TI ASC), which has one controller that controls 4 arithmetic pipelines, each having a 64-bit word length and 8 pipeline stages.

- From this data, we get K = 1, K' = 1, D = 4, D' = 1, W = 64, W' = 8 . So, we can represent TI ASC according to Handler's classification as follows :

TI ASC = <1*1, 4*1, 64*8> or <1, 4, 64*8>

Computer Performance: CPU execution time

- Computer performance is the total amount of work accomplished by a computer system.
- “How well the computer is doing the work it is supposed to do?”
- Computer performance is determined by the execution time
- Performance is inversely proportional to the execution time

$$\text{Performance} = \frac{1}{\text{Execution Time}}$$

Case: Processor A is faster than processor B

- It implies execution time of A is less than that of B
- Performance of A is better than that of B

Computer Performance: CPU execution time

Example: Machine A runs a program in 100 sec

Machine B runs a program in 125 sec

$$\text{Speedup} = n = \frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{Execution Time}_B}{\text{Execution Time}_A}$$

Therefore, Processor A is 1.25 times faster than processor B

The **total time** a CPU spends computing on a given task is called CPU time or CPU execution time

Computer Performance: CPU execution time

- **A program is comprised of a number of instructions executed , measured in: instructions/program**
- The average instruction takes a number of cycles per instruction (CPI) to be completed, measured in: cycles/instruction, CPI
- **CPU has a fixed clock cycle time C = 1/clock rate , measured in: seconds/cycle**
- CPU execution time is the product of the above three parameters as follows:

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

$$T = I \times CPI \times C$$

Case Study:

“How can we increase the performance on hardware and software while designing a processor?”

To improve the performance we can either

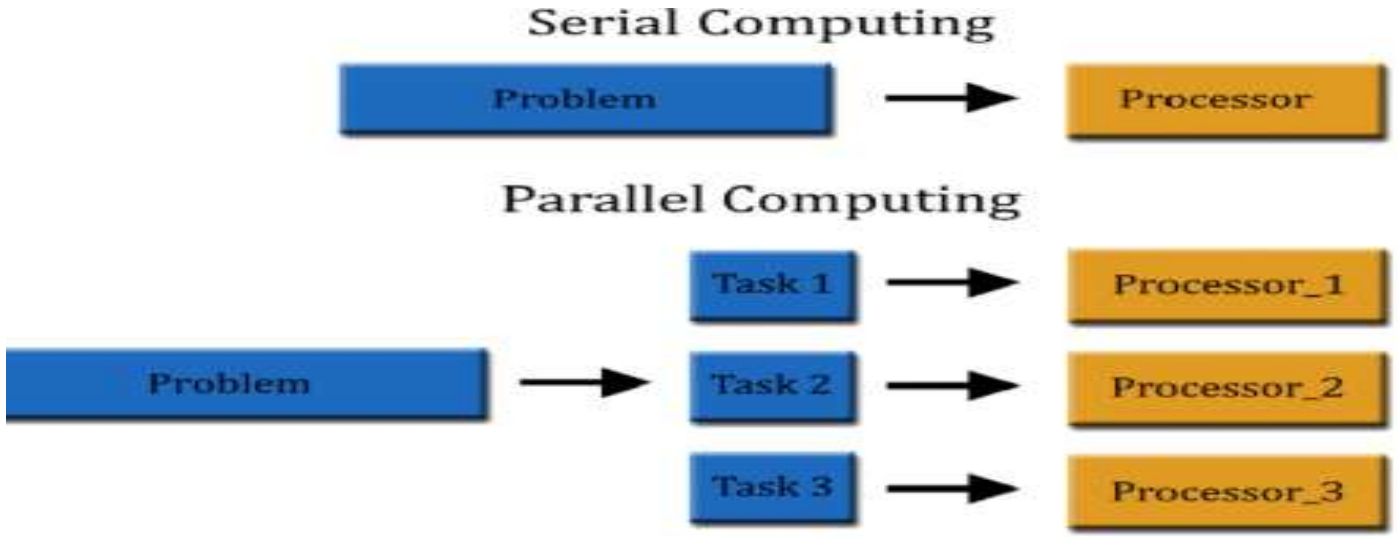
a) Decrease the clock cycles per instruction (CPI) by using new hardware

b) Decrease the clock time or increase the clock rate by reducing propagation delays or by use of pipelining

c) Decrease the no of required cycles or improve ISA or compiler

Parallelism

- Parallelism in computing is the process of breaking down a large task into smaller parts that can be processed simultaneously on multiple processors / CPU/ Cores
- In parallelism, we have different cores, different CPU, different processor and our application use that network to get the result
  - The goal of parallelism is to reduce the overall time it takes to complete a computation



Parallelism (Pipelining)

- Pipelining is a technique of decomposing a sequential process into sub-operation segments, where each segment perform its related task concurrently and transfers its result into next segment and at the end of the last segment, final result is obtained.
- Pipelining is the process of arrangement of hardware elements of the CPU such that its overall performance is increased.
- Simultaneous execution of one or more instruction takes place in pipelined processors.
- In pipelining multiple overall instruction are overlapped in execution

Performance Metrics for Parallel System

- Execution time

$$T_{total} = \text{p x } T_p$$

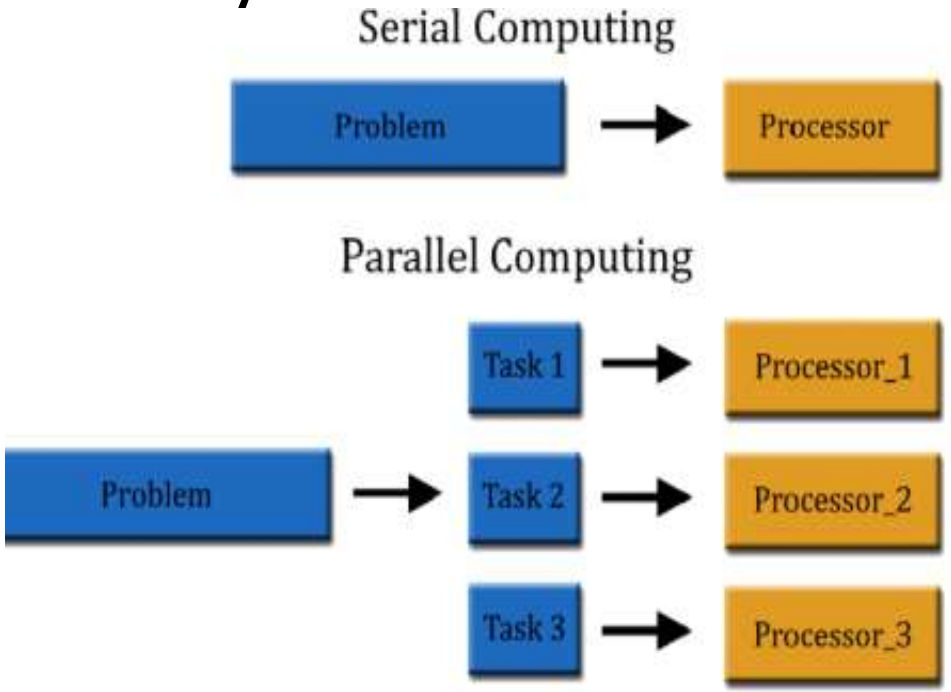
- Total Overhead

$$T_{overhead} = (\text{p x } T_p - T_s)$$

where, T_s → serial run time

T_p → parallel run time with 'p' processing elements

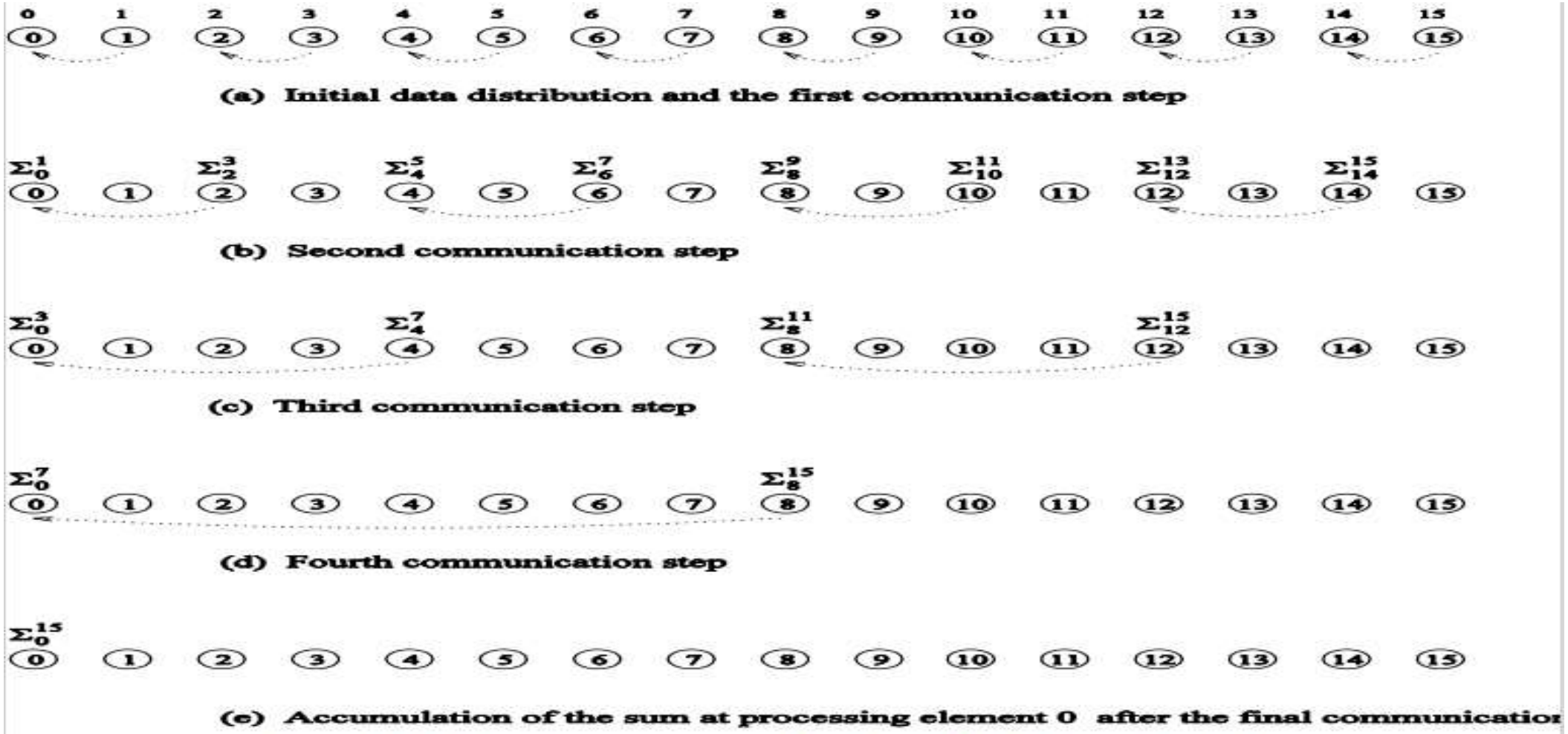
- Speedup: $S = \frac{Ts}{Tp}$



Adding n numbers using n processing elements

- It is defined as the ratio of the time taken to solve a problem on a single processing element to the time required to solve the same problem on a parallel computer with p identical processing elements.
- Consider the problem of adding n numbers by using n processing elements. Initially, each processing element is assigned one of the numbers to be added and, at the end of the computation, one of the processing elements stores the sum of all the numbers. Assuming that n is a power of two, we can perform this operation in logn steps by propagating partial sums up a logical binary tree of processing elements.
- The processing elements are labeled from 0 to 15. Similarly, the 16 numbers to be added are labeled from 0 to 15.

Adding n numbers using n processing elements



Computing the global sum of 16 partial sums using 16 processing elements . Σ^j_i denotes the sum of numbers with consecutive labels from *i* to *j*.

Performance Metrics for Parallel System

- **Speedup**
  - We denote speedup for the addition of n numbers is given as, S = $\frac{Ts}{Tp}$ = $\frac{\Theta(n)}{\Theta(Logn)}$ = Θ ($\frac{n}{Logn}$)

- **Efficiency**
  - Efficiency is a measure of the fraction of time for which a processing element is usefully employed
  - It is defined as the ratio of speedup to the number of processing elements. In an ideal parallel system, speedup is equal to 'p' and efficiency is equal to 1.
  - In practice, speedup is less than 'p' and efficiency is between 0 and 1, depending on the effectiveness with which the processing elements are utilized. We denote efficiency by the symbol E. Mathematically, it is given by

$$E = \frac{S}{p}$$

  - Therefore the efficiency of the algorithm for adding n numbers on n processing elements is

$$E = \frac{S}{p} = \Theta\left(\frac{\frac{n}{Logn}}{n}\right) = \Theta\left(\frac{1}{Logn}\right)$$

Performance Metrics for Parallel System

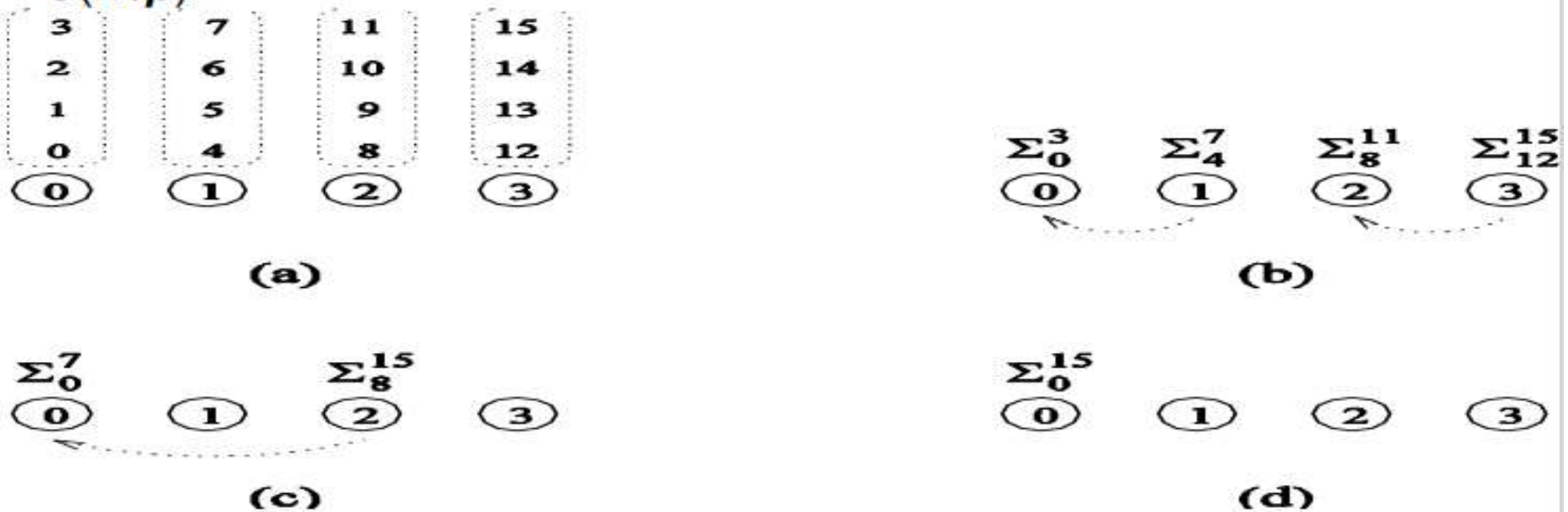
- The overall parallel execution time of this parallel system is $\Theta ((n/p) \log p)$
- The cost is $\Theta (n \log p)$, which is asymptotically higher than the $\Theta (n)$ cost of adding n numbers sequentially. Therefore, the parallel system is not cost-optimal

Can we build granularity in the above example in a cost-optimal fashion?

- Often, using fewer processors improves performance of parallel systems.
- Using fewer than the maximum allowable number of processing elements to execute a parallel algorithm is called scaling down a parallel system.
- A naive way of scaling down is to think of each processor in the original case as a virtual processor and to assign virtual processors equally to scaled down processors.
- Since the number of processing elements (or communication) decreases by a factor of n / p, the computation at each processing element increases by a factor of n / p.

Can we build granularity in the above example in a cost-optimal fashion?

- Each processing element locally adds its n / p numbers in time $\Theta(n / p)$.
- The p partial sums on p processing elements can be added in time $\Theta(n / p)$.



A cost-optimal way of computing the sum of 16 numbers using four processing elements.

Cost Optimal way

- The parallel runtime of this algorithm now is

$$\Theta\left((n/p) + \log p \right)$$

- The cost is $\Theta (n + p \log p)$
- As long as $n = \Omega(p \log p)$, the cost is $\Theta(n)$, which is the same as the serial runtime. Hence, this parallel system is cost-optimal.

Solve: A 4 stage pipeline has stage delay as 150, 120, 160 and 140 ns. Registers are used in between stages and have a delay of 5 ns. Assuming constant clock rate;

Find the total time to process 1000 data items on this pipeline

Hint: Speed up ratio (S) = $\frac{nt_n}{[K+(n-1)*t_p]}$

$$= \frac{total\ time\ required\ for\ n-task\ in\ non\ pipeline}{total\ time\ required\ for\ n-task\ in\ pipeline}$$

As the result of task increases, n becomes much larger resulting to n-1 to be n.

So, K+(n-1) => n

Hereby, maximum speed up ratio (S) = $\frac{ntn}{ntp} = \frac{t_n}{tp}$

where,

k = no of segments

t_p= time required for n task in pipeline

n = no of task/instructions

t_n = time required for n task in non pipeline

Case Study:

“Can we reduce the size of the instructions?”



Fig. Instruction set format

Parallel Programming Techniques

- There are two basic approaches in parallelizing the program as under
  - Explicit Parallel Programming
  - Implicit Parallel Programming

Explicit Parallel Programming

- Here the programmer develops a parallel program by explicitly specifying the parallelism.
- The programmer has to write down the parallel algorithm to solve the give program
- The techniques are:
  - Use of specific programming languages which supports parallel programming
  - Use of compiler directives for library function calls
  - Use of automatic parallelization tools that analyze the program and find parallelism

Implicit Parallel Programming

- In this technique, the programmer is relieved from parallelizing the program but it is the compiler which plays an important role.
- Here the compiler is responsible for extracting the parallelism in the program. There is no burden on the programmer to know about the parallel programming languages and also the underlying parallel architecture.
- We shall be considering only the explicit programming approach here.

Parallel Programming Models

Computer

- Sequential computer
  - uniprocessor
- Parallel Computer
  - Shared Memory (common memory, multiprocessor)
    - UMA (uniform memory access)
    - NUMA (non uniform memory access)
      - Shared local memory
      - Hierarchical cluster
    - COMA (cache only memory access)
  - Unshared Memory (distributed memory, multicomputer, message passing multicomputer)
    - NORMA (no remote memory access)

UMA model for multiprocessor system

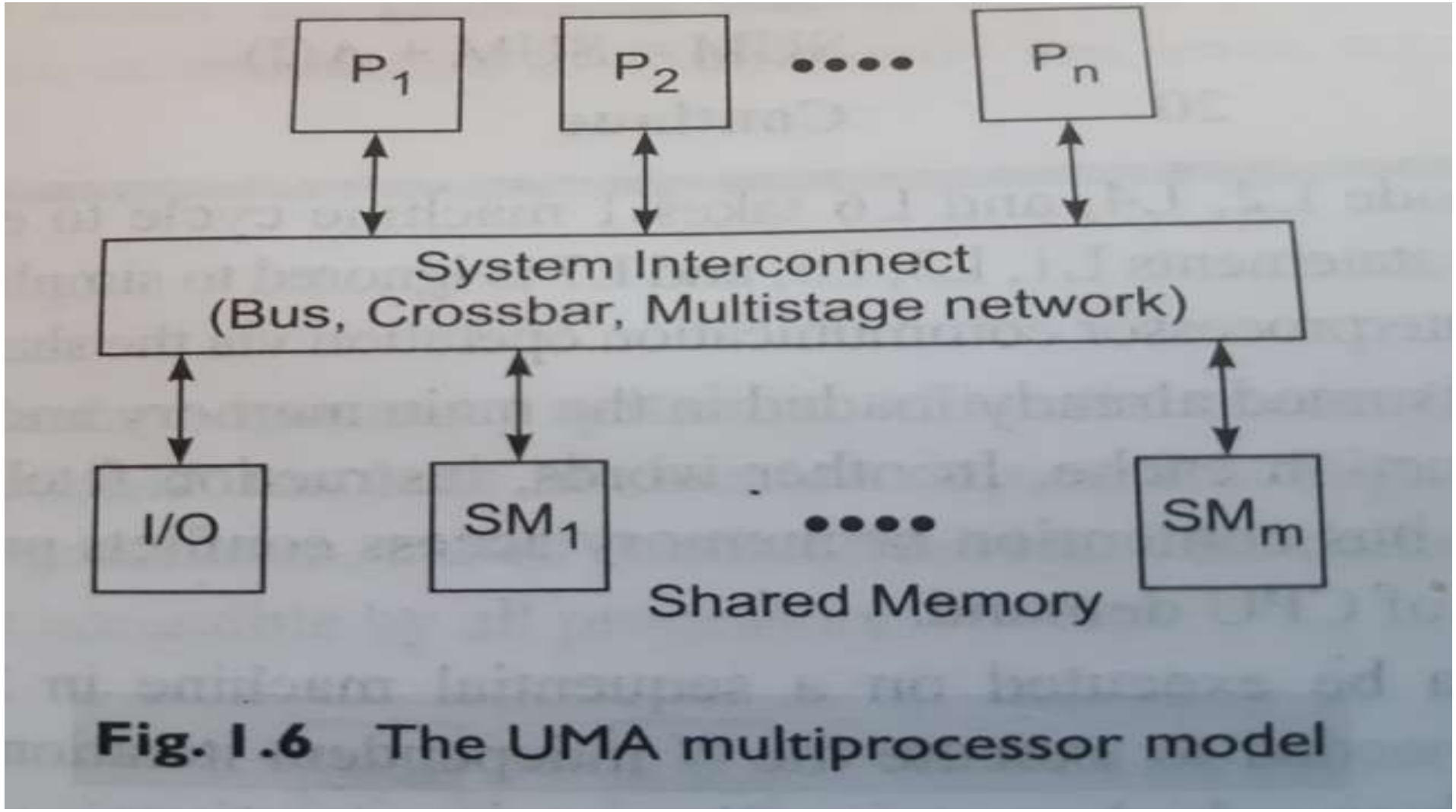
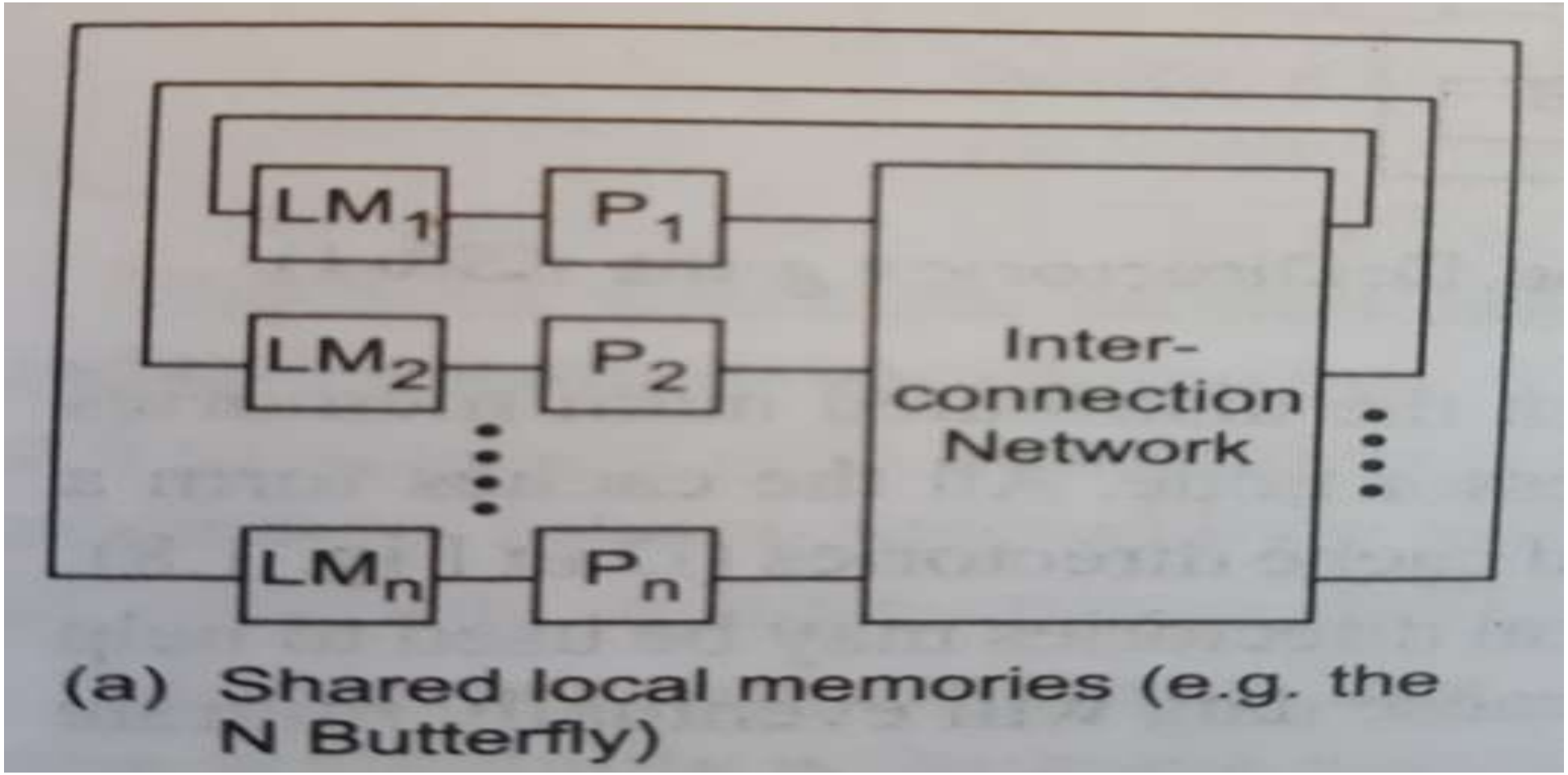


Fig. 1.6 The UMA multiprocessor model

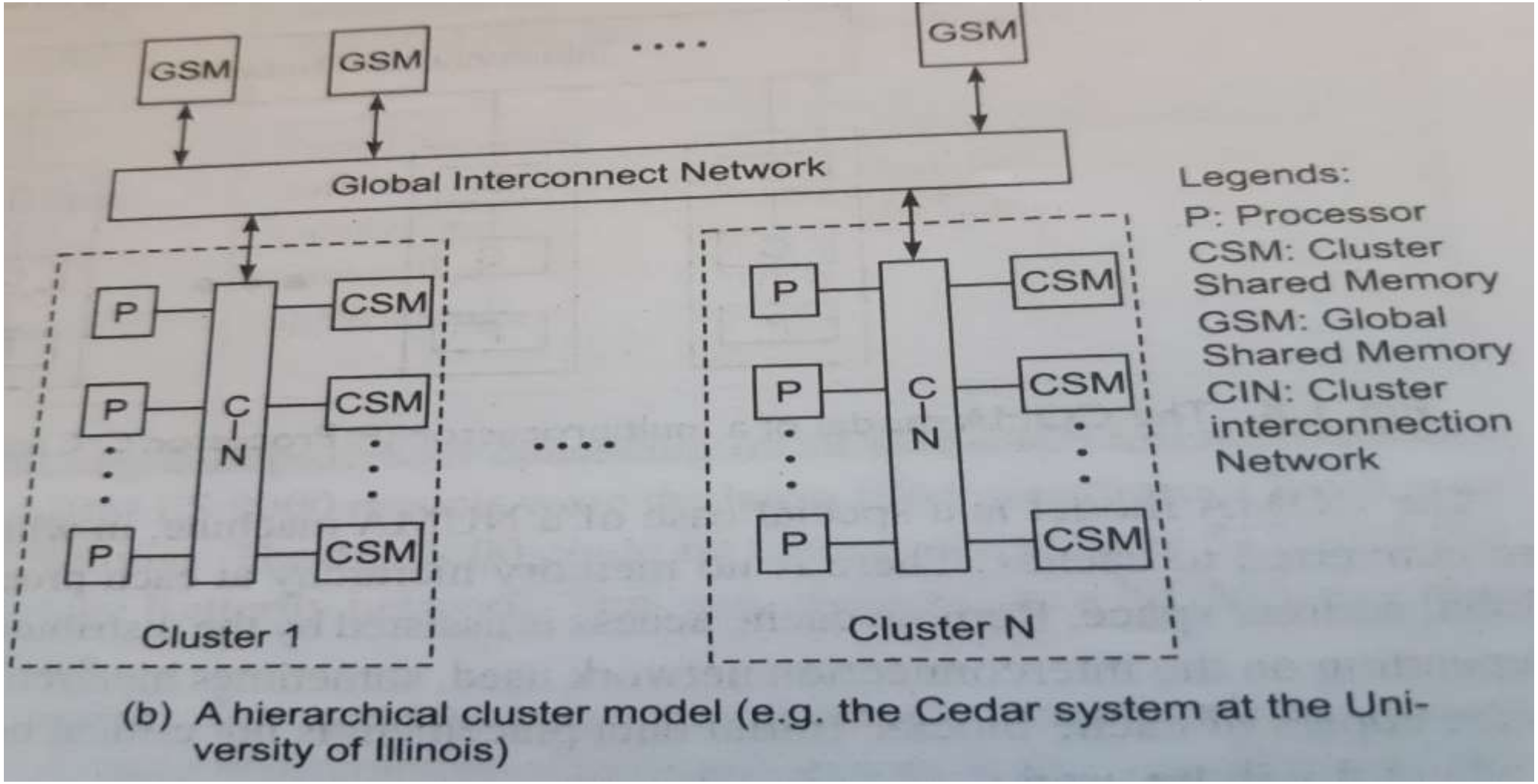
From kai Hwuang, 3e, p19

NUMA model for multiprocessor system



From kai Hwuang, 3e, p21

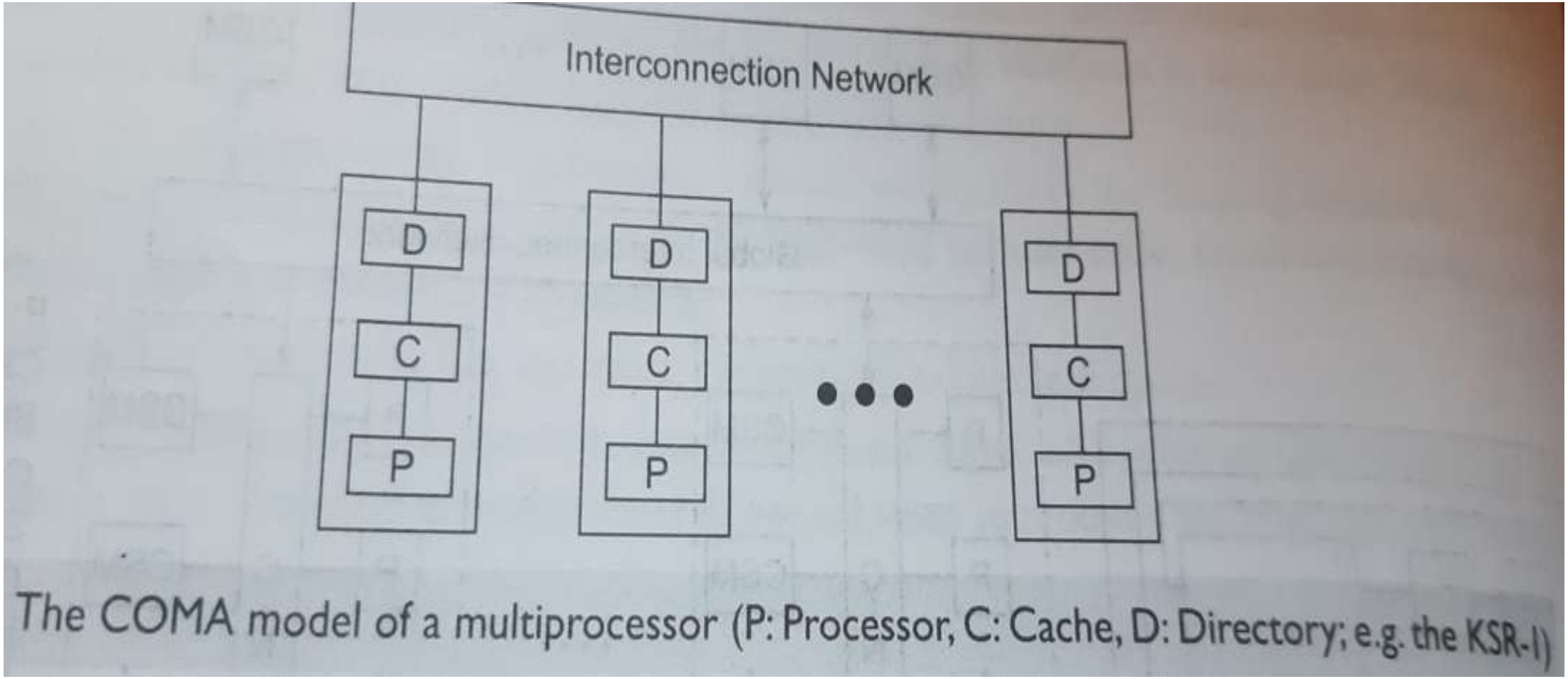
NUMA model for multiprocessor system



(b) A hierarchical cluster model (e.g. the Cedar system at the University of Illinois)

From kai Hwuang, 3e, p21

COMA model for multiprocessor system



The COMA model of a multiprocessor (P: Processor, C: Cache, D: Directory; e.g. the KSR-1)

From kai Hwuang, 3e, p22

Distributed Memory Multicomputer

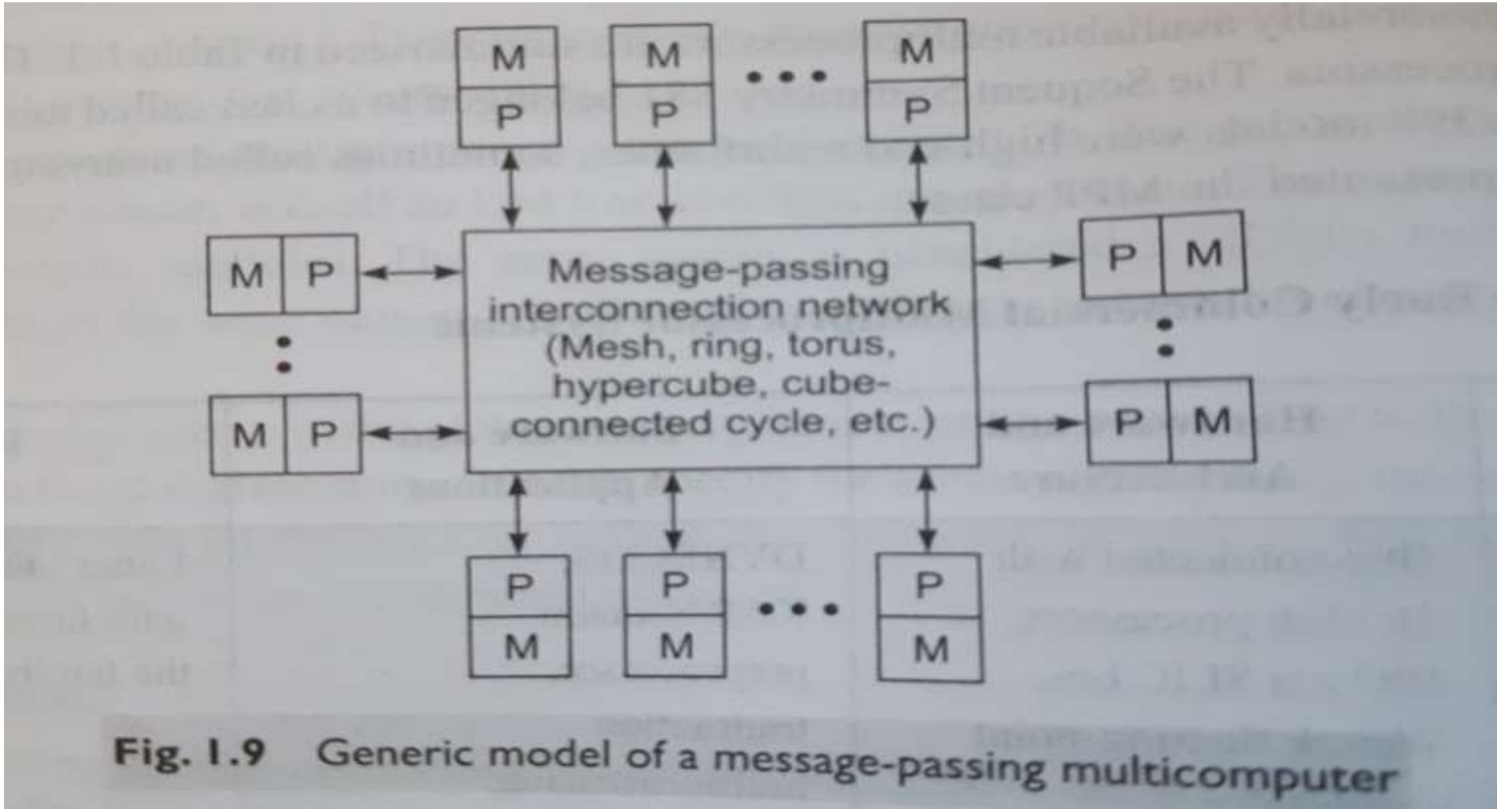


Fig. 1.9 Generic model of a message-passing multicomputer

From kai Hwuang, 3e, p24

Classification based on memory arrangement

- Parallel architectures can be classified into two major categories in terms of **memory arrangement**:
  - **Shared memory**
  - **Distributed memory** or message passing
- This classification constitutes a subdivision of MIMD parallel architecture and are also known as:
  - *Shared memory* architecture → *tightly coupled* architecture
  - *Distributed memory* architecture → *loosely coupled* architecture

- A third choice is somewhere in between: groups of processors share a memory block while the different groups, also referred to as *nodes*, have distinct memory blocks

Shared Memory Multiprocessor

- An **UMA** architecture comprises two or more **processors with identical characteristics**
- The processors:
  - share the same main memory and I/O facilities
  - are interconnected by networks as some form of bus-based, crossbars, or multiport memories

- Processors perform the same functions under control of an operating system, which provides interaction between processors and their programs at the job, task, file and data element levels

Shared Memory Multiprocessor

- In the case of **NUMA** architectures, memory is *physically distributed* but *logically shared*
- Each processor has part of the shared memory attached, but the memory has a single address space, therefore, any processor could access any memory location directly
- The **memory access time** of processors **differs** depending on which region of the main memory is accessed

- A subclass of NUMA system is **cache coherent NUMA** (cc-NUMA) where cache coherence is maintained among the caches of various processors
- The main advantage of a cc-NUMA system is that it can deliver effective performance at higher levels of parallelism

Message Passing Multicomputer

- In a **distributed memory architecture** each unit is a complete computer building block including the processor, memory and I/O system
- Units are referred to as ***nodes***, and there is no sharing of memory between them
- Nodes are typically able to store messages in buffers (temporary memory locations where messages wait until they can be sent or received), and perform send/receive operations at the same time as processing
- Hence, communication among the processors is established in the form of I/O operations through message signals and interconnection networks

Message Passing Multicomputer

- **Example**
  - If a processor needs data from another processor
  - It sends a signal to that processor through an interconnection network demanding the required data
  - The remote processor then responds accordingly

- Notice that, the further the physical distance to the remote processor, the longer it will take to access the remote data
- The processing units of a message passing system may be connected in a variety of ways ranging from architecture-specific interconnection structures to geographically dispersed networks
- The message passing approach is, in principle, scalable to large proportions

Example: Sum of array elements

Pseudo code for sequential program

```
sum = 0;
int A[maxsize];
int main()
for(i=0; i<maxsize; i++)
{
        sum +=A[i];
}
print(sum);
```

Example: Sum of array elements

Pseudo code for parallel algorithm using Message Passing

```
//shared varibales:
sum = 0;
int A[maxsize];
int lockvar;
int joinvar;

//private varibale
int lo, hi, mysum=0;
lo = pid * maxsize/nproc;
// nproc is the number of processes/threads
// pid = process ID, o to nproc-1
hi = lo + maxsize/nproc;
for(i=lo; i<hi; i++)
{
        mysum += A[i];
}
BARRIER (joinvar);
LOCK(lockvar);
 sum +=mysum;
 UNLOCk (lockvar);
```

Example: Sum of array elements

Pseudo code for sequential program explained

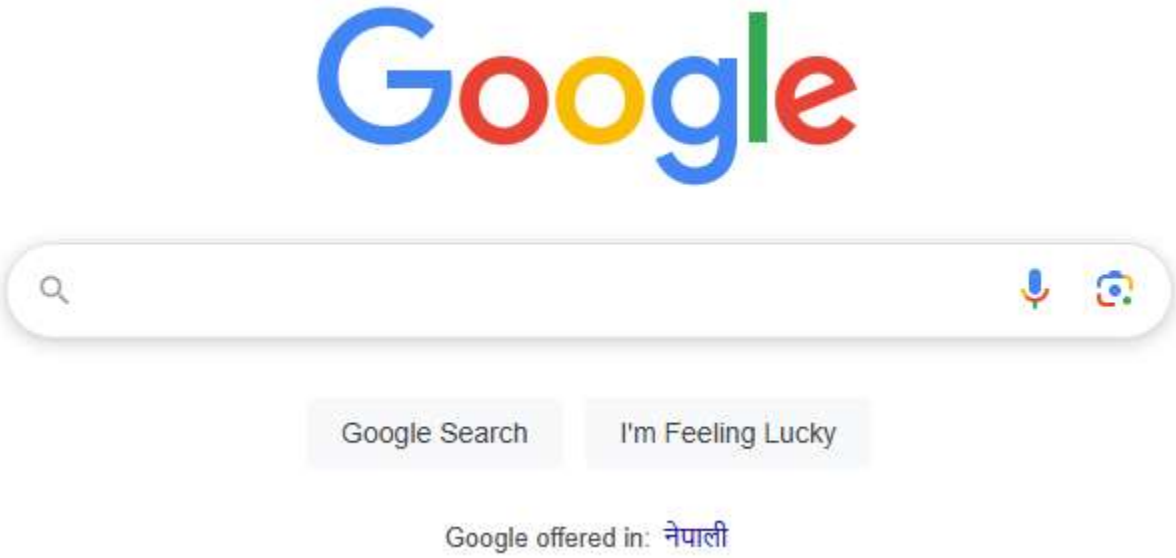
- Variables declared globally can be shared
- Array, lock-var and join-var are accessible to all threads
- Each thread computes the local sum in mysum
- Since values maybe different with each thread, use critical section to add mysum in serial fashion
- All all the threads to complete local summation before adding to global.
- For barrier synchronization – variable is joinvar
- Barrier makes the thread to wait unitill all threads have reached the barrier
- Barriers and locks can be implemented using Load, Store and Read-modify-Writ instructions in the ISA

Parallel Programming

- **OpenMP (C, C++ & Fortran):**
  - For distributed memory and shared memory systems we normally use C and its extension
- **POSIX threads: pthreads**
  - For shared memory based communication we can create threads using pthreads that is p thread library
  - Library support is required for doing this
- **MPI (Message Passing Interface):**
  - For message passing, the MPI interface is very popular and is used in shared as well as distributed memory system
  - Implementation is based on libraries, functions and macros
- **CUDA and OpenCL:**
  - GPU programming for parallel processing.
- **Java Parallel Streams:**
  - Multithreading using Java's parallel stream APIs.
- **Python:**
  - Libraries like multiprocessing, Dask, and Ray for parallelism.

Case Study: deadline 26th December

- What makes Google the best search engine? Explain the role of parallel processing in enabling Google to handle large-scale web searches efficiently.



Google’s Algorithm

- Google algorithm is a program used by Google for ranking websites on its result pages among organic result
- To list a few basic types of Google algorithms, we have
  - **Page Rank Algorithm**
    - It is the heart of google searching
    - Developed by Larry Page and Sergy Bin
    - Based on the hyperlinks map
    - An excellent way to prioritize the results of the web keyword searches
  - **Panda Algorithm**
    - Launched on Feb 2011
    - Its main target was to detect “content farms” and to block them from showing in Google search results.
    - A content farm or content mill is a company that employs freelance creators or uses automated tools to generate a large amount of web content which is specifically designed to satisfy algorithms for maximal retrieval by search engines, known as SEO (search engine optimization)

Google’s Algorithm

- **Penguin Algorithm**
  - Google Penguin was launched in April 2012
  - Fighting against spam sites and erasing them
  - A search engine update that aims to reduce web spam and improve the quality of search results:
  - Basically it is a place where all your hard work is stored
- **Humming code Algorithm**
  - Launched in Sept 2013
  - was aimed to improve the indexing of information rather than sorting through information.
  - The name originates from being precise and fast
  - It made a biggest change in Google history
  - This algorithm doesn’t effect SEO

Google’s Algorithm

- **Pigeon Algorithm**
  - This update was released on 2014
  - Was designed to improve ranking parameters based on distance and location
  - Aimed to increase the ranking of local listing in a search
  - Here you need to check local.google.com and see whether your business is listed already or not, follow the online instruction and select a verification method.

- **Google Fred Algorithm**
  - Updated on March 2017
  - Designed by Google to target black-hat tactics that are connected to overly aggressive monetization
  - Google Fred specifically look s for excessive ads, low value content and websites that generally offer very little user benefit

Google’s Algorithm

2019 Update : Broad Core Algorithm

- It is all about providing a better user experience
- It is believed that Google makes tweaks to its algorithm at least twice per day, which means 600 to 700 updates a year
- Google updates are “broad” in the sense that they don’t target any specific. Rather they are designed to improve Google’s system overall
- So the pages that drop in ranking are not being penalized, they are being reassessed against other web content that has been published since the last update

……

……..

……..

Case Study:

- Understanding Parallel Processing in Google:
  - Explain the role of parallel processing in enabling Google to handle large-scale web searches efficiently.
  - How does dividing tasks into smaller sub-tasks and processing them concurrently improve the speed and reliability of search results?

- Major Advancements:
  - Discuss how Google's PageRank algorithm uses parallel processing to rank web pages.
  - Explore the MapReduce framework and its significance in processing massive datasets. How does this framework utilize parallel processing to simplify complex computational tasks?

- Role of Machine Learning:
  - How has the integration of machine learning enhanced Google's search engine capabilities? Highlight the use of neural networks in understanding user intent and improving search relevance.

- Comparative Analysis:
  - Compare Google’s approach to parallel processing with another search engine (e.g., Bing or Yahoo). What unique strategies make Google superior?

- Engineering Insights:
  - As an ME engineering student, suggest how principles of parallel processing could be applied to other fields of engineering, such as mechanical systems simulation or optimization

This case study encourages critical thinking and interdisciplinary application of parallel processing concepts

Set A

1. Explain the significance of parallel processing in modern computing. How does parallelism improve system performance, and what are the key challenges associated with implementing parallel algorithms?
2. Why is parallel programming essential in modern computing? Discuss the key characteristics of parallel algorithms and explain how they influence the design and efficiency of parallel systems.
3. Explain the architectural classification of computer systems based on Flynn's taxonomy. Provide examples for each category and discuss their relevance in parallel computing.
4. Is it possible to develop a parallelizing compiler that transforms a sequential program into a multithreaded program? Provide justification for your answer.

Set B

1. In the example of adding 'n' numbers using 'p' processing elements, can granularity be achieved in a cost-optimal manner? Provide justification for your answer.
2. Differentiate between data and instruction. Compare and contrast between Von-Neumann and Harvard architecture.
3. "Is it possible to implicitly reduce the size of an instruction? Explain your reasoning."
4. What are the different parallel programming models used in parallel computing? Explain each model with examples and discuss how they impact the performance and scalability of parallel systems.